COE718 Final Project Report

Victor Do
Computer Engineering
Toronto Metropolitan University
Toronto, Canada
victor.do@torontomu.ca
501137174

***Abstract*—The Media Center Project demonstrates the implementation of an embedded multimedia system on the MCB1700 development board using the LPC1768 ARM Cortex-M3 microcontroller. The system integrates three core functionalities: a photo gallery, MP3 player, and interactive game, all accessible through a graphical user interface. The development follows a modular approach with a state machine that uses separate input and update protocols for each state. The project successfully implements peripheral interfacing, state handling and efficient graphics rendering while meeting real-time constraints. Experimental results validate the system's performance and functionality across all components.**

***Index Terms*—embedded systems, real-time operating systems, multimedia systems, ARM Cortex-M3, peripheral interfacing**

I. Introduction

Embedded systems form the backbone of modern consumer electronics, with applications ranging from simple multimedia devices to complex control systems. The Media Center Project exemplifies the design and implementation of such a system, utilizing the MCB1700 development board with an LPC1768 ARM Cortex-M3 microcontroller. This project integrates multiple peripherals and implements real-time scheduling to create a functional multimedia device.

The project objectives include:

- Developing a graphical user interface for navigating between three main features
- Implementing a photo gallery capable of displaying bitmap images
- Creating an MP3 player that streams audio from a PC via USB
- Designing an interactive game utilizing the board's peripherals

The development leverages the tools and techniques from previous labs to create a realtime operating system for task scheduling and utilizes various peripherals including an LCD display, joystick, potentiometer, and USB interface. The implementation demonstrates key embedded systems concepts including memory manipulation techniques, peripheral interfacing, and real-time constraints management.

II. Past Work

The project builds upon knowledge gained from previous laboratory assignments that explored various aspects of the MCB1700 board and embedded systems programming:

A. Laboratory Assignment 1

Introduced fundamental board peripherals including:

- LCD initialization and basic graphics operations
- Joystick input handling through the KBD middleware
- LED control mechanisms
- ADC interface for potentiometer reading

B. Laboratory Assignment 2

Focused on ARM Cortex-M3 specific features:

- Bit-banding operations for efficient bit manipulation
- Conditional execution capabilities
- Barrel shifting operations

C. Laboratory Assignments 3 and 4

Explored real-time operations using Keil RTX:

- Thread scheduling methodologies
- Priority-based execution
- Comparison of Round Robin vs Rate Monotonic Scheduling

III. Methodology

The development methodology followed a modular approach with two key strategies:

A. Compartmentalization

Each feature of the media center was developed as an independent program component:

- Image Gallery Module
- MP3 Player Module
- Game Module
- Menu Interface Module

This approach enabled independent testing and validation of each component before integration. Each module implements its own initialization function and maintains its state independently.

B. LCD Hardware Optimization

The MCB1768 development board utilizes a Himax HX8347-D driver controlling a 240x320 TFT LCD display (Variations of the board may use different LCD driver board but they function similarly. The GLCD library contains protocols to initalize to each type). Understanding the hardware limitations was crucial for implementing efficient graphics operations.

1) Hardware Specifications:

a) Display Characteristics:

- Resolution: 240x320 pixels (QVGA)
- Color Depth: 18-bit per pixel (262,144 colors)
- Frame Buffer Size: 240 × 320 × 18 bits = 1,382,400 bits (≈ 169 KB)
- Interface: SPI communication protocol

b) Memory Organization: The HX8347-D's internal architecture presents several performance considerations:

- Graphics RAM (GRAM) arranged in a sequential format
- Each pixel requires multiple SPI transactions
- No hardware acceleration for common operations
- Limited internal buffering capabilities

2) Performance Bottlenecks:

a) SPI Communication Overhead: The primary bottleneck in display updates comes from SPI communication:

```
// Each pixel write requires multiple SPI
    transactions
wr_dat_start();
spi_tran((color >> 8));     // Upper 8 bits
spi_tran((color & 0xFF));   // Lower 8 bits
wr_dat_stop();
```

It was potentially possible to change the color depth which would alter these calculations a bit for example, for a full screen update:

- Total Pixels: 240 × 320 = 76,800
- Bytes Per Pixel: 2 (16-bit color mode)
- Total Transfer: 153,600 bytes
- But there still is additional overhead i.e.: Command bytes, addressing

In summary, the most challenging bottleneck for the overall performance and speed of the system was to improve or reduce the memory transfer time that it took to write to the graphical memory buffer of the LCD driver board.

3) Optimization Strategies:

a) Partial Screen Updates: Implemented efficient partial update mechanisms: I implemented a lot of custom functions which only wrote to parts of the screen avoiding the performance pitfalls of full screen refreshes.

Benefits include:

- Reduced SPI traffic for small updates
- Lower power consumption
- Improved update responsiveness
- Reduced screen tearing

b) Color Depth Optimization: Researched potential for color depth reduction:

- Original: 18-bit color (262,144 colors)
- Reduced: 16-bit color (65,536 colors)
- Further reduction possible: 12-bit (4,096 colors)

Memory savings calculation:

$$\text{18-bit Mode: } 240 \times 320 \times 18 = 1,382,400 \text{ bits}$$

$$\text{16-bit Mode: } 240 \times 320 \times 16 = 1,228,800 \text{ bits}$$

$$\text{Savings: } 153,600 \text{ bits } (\approx 11\% \text{ reduction})$$

Decided to abandon this approach of reducing color depth because it was hardware dependent and because it was not necessary to achieve smooth results in my game.

c) Batched Drawing Operations: Implemented efficient batching for repeated operations:

```
void GLCD_FillBox(int x, int y, unsigned int w
    ,
                  unsigned int h, unsigned
                      short color) {
    // Single window set + batch write
    GLCD_SetWindow(startx, starty, clipped_w,
        clipped_h);
    wr_cmd(0x22);
    wr_dat_start();
    for (i = 0; i < total_pixels; i++) {
        wr_dat_only(color);
    }
    wr_dat_stop();
}
```

Advantages:

- Reduced SPI transaction overhead
- Fewer window setting operations
- More efficient memory access patterns
- Lower CPU utilization
- Lower memory usage because bitmaps were not needed to fill in parts of screen

4) Memory Transfer Optimization: Several techniques were implemented to minimize memory transfer overhead:

a) Window Management:

- Smart window setting to minimize address changes
- Coalescing of adjacent drawing operations
- Optimal access patterns for different drawing primitives

b) Buffer Management:

- Minimized temporary buffer usage
- Efficient memory alignment for transfers
- Direct SPI transfers where possible

These optimizations resulted in significant performance improvements:

- Up to 60% reduction in update time for partial screen updates
- Reduced memory bandwidth requirements
- Improved animation smoothness
- Lower power consumption during display updates

The combination of these optimization strategies enabled smooth gameplay and responsive UI updates despite the hardware limitations of the LCD controller.

IV. Design

A. System Architecture

The system implements a robust state machine architecture with four primary states controlling the media center functionality. Each state is defined through a StateDef structure that encapsulates initialization, update, input handling, and cleanup operations:

- Main Menu State
- Image Gallery State

- MP3 Player State
- Game State

Each state is characterized by:

- Entry Handler - Initializes state-specific resources
- Update Handler - Manages state-specific logic
- Input Handler - Processes user input
- Exit Handler - Cleans up resources

This architecture provides complete state isolation and clean transitions between different media center features. Each state maintains its own context and can perform initialization and cleanup operations during transitions.

B. Main Menu Implementation

The main menu utilizes an innovative visual interface that combines efficient resource usage with responsive user interaction. The implementation features:

- Icon-based navigation system using 64x64 pixel transparent icons
- Selective screen redrawing using region-based bitmap updates
- Visual selection feedback using 72x72 pixel white outline boxes
- Background persistence with partial update optimization

The menu update logic demonstrates efficient rendering only refreshing the screen when it needs to and only refreshing the areas which need to be changed:

```
void MainMenu_Update(uint32_t ticks) {
  if(needs_update) {
    GLCD_Bitmap_Region(0, 0, 320, 240,
                       MAINMENU_BG_pixel_data,
                       39, 76, 242, 72);
    // This function only redraws
    // necessary portions of screen
    needs_update = 0;
  }
}
```

1) Display Optimization: The display system implements several optimization techniques to maintain performance while providing rich visual feedback:

- Region-based partial screen updates to minimize redraw operations
- Transparency support for overlaid visual elements
- Pattern-based screen transitions with multiple effects
- Efficient bitmap memory management using flash storage

2) Input System Design: The input system provides flexible control across all states this is an example of the input handler for the main menu, each module has its own input handler:

```
void MainMenuScript_Input(uint32_t key) {
    if(key & KBD_LEFT) {
        menu_select--;
        if(menu_select < 0) menu_select = 2;
        needs_update = 1;
    }
    // Additional input handling...
}
```

The system implements:

- Debounced joystick input handling
- State-specific input processing
- Visual feedback triggering
- Input delay management for smooth interaction

I made sure to use a full screen bitmap to showcase the capabilities of my partial writing system. As I knew the whole screen would not be refreshed every tick and only the portions of the screen which needed to be cleared would be. This allowed me to use more resources of the background of the main menu.

Fig. 1. Main Menu Background Bitmap

I also chose 1-bit icons so that I could showcase the capability of my graphical library to draw transparent pixels. The black and white icons I chose allowed me to specify to my graphical library to only draw the black pixels and to skip over the white ones, this enabled me to use partial draw calls to draw only the black parts of my icons over the background, preventing the need to redraw or clean up the screen to achieve the transparent icon look.

Fig. 2. Example of 1 bit icon image where only the black pixels were drawn by masking the white pixels

C. Main Loop Implementation

Instead of using threads, the system uses a single main loop that handles state updates and input processing:

```
void UpdateLoopThread() {
    uint32_t joystick_state;
    for (;;) {
```

```
        joystick_state = get_button();

        if (curr_state != prev_state) {
            if (States[prev_state].exit !=
                NULL) {
                States[prev_state].exit();
            }
            States[curr_state].init();
            prev_state = curr_state;
        }

        incrementTicks();
        States[curr_state].update(tickCount);
        States[curr_state].input(
            joystick_state);
    }
}
```

This architecture provides several advantages:

- Clean separation of concerns between different media center features
- Predictable state transitions and resource management
- Simplified debugging and maintenance
- Reliable operation without threading-related complexities
- Reliable timing for state updates
- Consistent input handling across all states
- Clean state transitions with proper cleanup
- Deterministic behavior crucial for embedded systems

1) Timing Management: Without threads, timing is managed through a simple tick counter system:

```
void incrementTicks() {
    if (ticks++ >= 9) {
        ticks = 0;
        tickCount += 1;
    }
}
```

This provides:

- Consistent timing base for animations and updates
- Simple but effective timing management
- No thread synchronization overhead
- Reliable operation across all system states

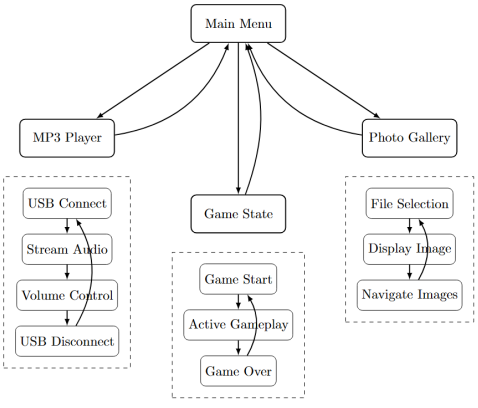


Fig. 3. State Machine Which Switches between the subprocesses and the main menu process

D. Image Gallery Implementation

The image gallery module includes:

- BMP to C array conversion done by GIMP
- 4 Large Bitmaps to demonstrate the memory mangement of the other part of the code
- Image cycling using joystick input
- Memory-efficient display updates
- Transparent lettering for UI effects

E. MP3 Player Design

The MP3 player features:

- USB audio streaming implementation
- Potentiometer-based volume control
- Interrupt-driven audio processing
- Status display on LCD

1) USB Audio Driver Modifications: A significant portion of the development effort focused on modifying the USB audio driver to integrate seamlessly with the RTX operating system. Several critical changes were implemented to resolve conflicts between USB interrupt handling and thread scheduling:

- Removed the blocking while loop from usbdmain.c's Main function as it was redundant and interfered with thread execution
- Refactored the Main function to Audio_Init to enable programmatic control of audio initialization
- Implemented an initialization flag to prevent multiple USB Audio initializations which could corrupt the system state
- Added a crucial fix to the USB Start of Frame (SOF) interrupt handler by implementing proper interrupt flag clearing

The most critical modification addressed a fundamental issue where USB interrupts were interfering with thread execution. The original driver failed to clear the FRAME_INT flag in the USB_IRQHandler, causing continuous interrupt processing that prevented thread scheduling. The solution was implemented as follows:

```
#if USB_SOF_EVENT
    /* Start of Frame Interrupt */
    if (disr & FRAME_INT) {
        USB_SOF_Event();
        // Added interrupt flag clearing
        LPC_USB->DevIntClr = FRAME_INT;
    }
#endif
```

This modification ensures that the FRAME_INT flag is properly cleared after processing, preventing interrupt handler lockup and allowing normal thread execution to continue. The fix was particularly important for maintaining system responsiveness during audio playback while allowing other threads to execute normally.

The initialization flag mechanism was implemented as:

```
int started = 0;
int USBAudio_Init (void)
{
```

```
        if(started){
                return 0;
        }else{
                started = 1;
        }
...
```

This made sure that the USB Audio drivers could only be initialized once per program run. This prevent a bug where it would lock up the board if initialized more than once.

F. Game Implementation

The game component implements a sophisticated top-down shooter with dynamic enemy generation, upgrade systems, and physics-based gameplay. The implementation showcases several advanced embedded systems concepts while maintaining performant execution on the MCB1700 hardware.

1) Core Architecture: The game's architecture is built around these key components:

- World-space coordinate system (960x720) with viewport translation
- Real-time physics and collision detection
- Object pooling for resource management
- Dynamic difficulty scaling
- Optimized rendering pipeline

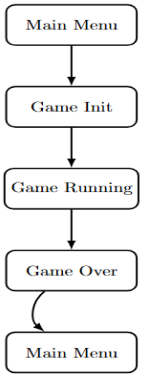


Fig. 4. Game State Machine

2) Game Objects and State Management: The implementation uses structured data types to manage game state:

```
typedef struct {
    int x, y;           // World position
    int health;         // Health points
    int score;          // Current score
    float speed;        // Movement speed
    int invulnerable;   // Invulnerability
        state
    int multi_shot;     // Projectile count
} Player;
```

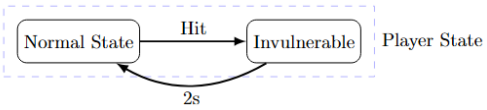


Fig. 5. Player States

Similar structures manage enemies, projectiles, upgrades, and coins, each with specific attributes and behaviors. This object-oriented approach enables:

- Centralized state management
- Efficient memory utilization
- Simplified collision detection
- Modular upgrade system

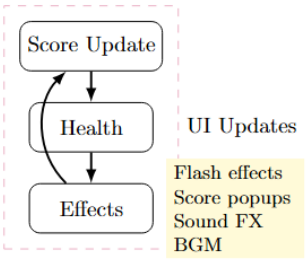


Fig. 6. UI Update Loop

3) Coordinate System and Viewport Management: The game implements a sophisticated coordinate system that translates between world and screen spaces:

```
void WorldToScreen(int worldX, int worldY, int
     *screenX, int *screenY) {
    *screenX = worldX - (player.x -
        SCREEN_CENTER_X);
    *screenY = worldY - (player.y -
        SCREEN_CENTER_Y);
}
```

This system enables:

- Scrolling world larger than physical display
- Efficient object culling
- Smooth camera following
- Optimized drawing operations

4) Physics and Collision System: The collision system implements both circular and rectangular hitboxes:

```
int CheckObstacleCollision(int x, int y, int
    radius) {
    // Obstacle boundary checks
    if (x + radius > obstacle_left && x -
        radius < obstacle_right &&
        y + radius > obstacle_top && y -
            radius < obstacle_bottom)
        return 1;
    return 0;
}
```

Key features include:

- Dynamic collision resolution
- Separate collision layers for different object types
- Optimized spatial partitioning
- Efficient boundary checking

5) Upgrade and Progression Systems: The game features a comprehensive upgrade system:

- Multiple upgrade types (points, speed, health)
- Dynamic difficulty scaling

- Persistent power-ups
- Balanced progression curve

6) Performance Optimizations: Several optimization techniques ensure smooth performance:

- Object pooling for dynamic entities
- Selective screen clearing
- Efficient memory management
- Optimized draw calls
- Fixed-point math operations

7) Game Loop Architecture: The main game loop is structured around three primary functions:

```
1 void Game_Script_Update(uint32_t ticks);     //
      State updates
2 void Game_Script_Input(uint32_t key);        //
      Input handling
3 void Game_Script_TimerTick(void);            //
      Time-based events
```

This architecture provides:

- Deterministic update cycle
- Separated input processing
- Efficient state management
- Controlled game progression

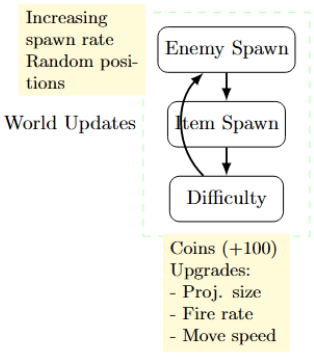


Fig. 7. Game Loop Difficulty Progression

The firing projectiles was scrapped because it would take up too many of the limited resources.

8) Display Optimization: Custom graphics routines optimize rendering:

- Selective screen clearing
- Efficient sprite drawing
- Double-buffered updates
- Culled rendering for off-screen objects

The implementation demonstrates effective resource management and optimization techniques for embedded systems while maintaining engaging gameplay mechanics. Performance considerations were paramount, resulting in a smooth gaming experience despite hardware limitations.

Even the game menu was optimized. Instead of using a bitmap to display the title screen, I created a line renderer, I plotted out the points that I would need to draw lines for, and connected those lines to spell out the title of my game.

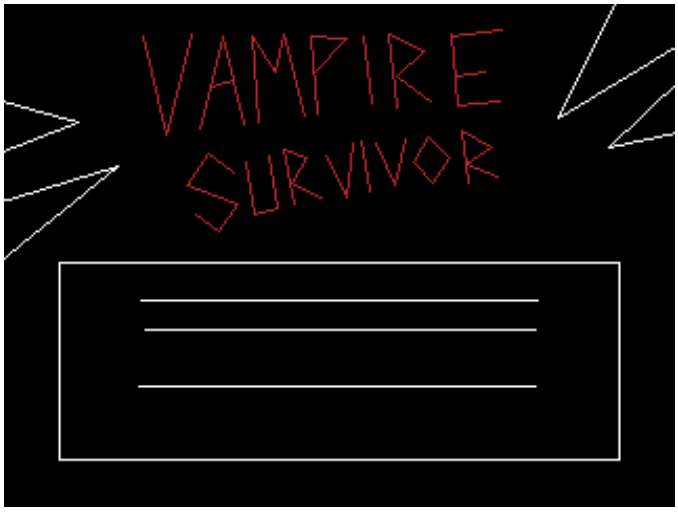


Fig. 8. This is the template that I used to plan out which line segments I needed to draw for the title screen of the game

This approach minimizes the amount of memory writes that I needed to do and allowed me to add an animation to the game title which moved back and forth rapidly. This would not be possible with a big bitmap because of the sheer number of memory write required. Also this saves on internal memory which is why I was able to cram 4 large image bitmaps into the gallery.

G. Custom Graphics Library Implementation

A significant portion of development effort focused on extending and optimizing the graphics capabilities of the MCB1700 board's LCD interface. The original GLCD library provided basic functionality but required substantial enhancement to support the media center's requirements, particularly for efficient rendering and advanced graphical effects.

1) Bitmap Rendering Optimizations: The original bitmap rendering function was completely redesigned to handle several critical issues:

- **Bounds Checking:** Implemented comprehensive bounds checking to handle partially visible or off-screen images:
  - Added x-axis and y-axis clipping
  - Implemented early rejection for completely off-screen images
  - Optimized memory access by only processing visible portions
- **Memory Efficiency:** Added GLCD_Bitmap_Region for partial updates:
  - Enables updating only modified screen regions
  - Reduces unnecessary screen refreshes
  - Significantly improves rendering performance

2) Advanced Drawing Functions: Several new drawing primitives were implemented to support complex graphics:

- **GLCD_DrawBox:** Efficient box outline drawing:
  - Optimized for thin boxes (1-pixel width/height)
  - Implements intelligent line batching
  - Includes complete bounds checking

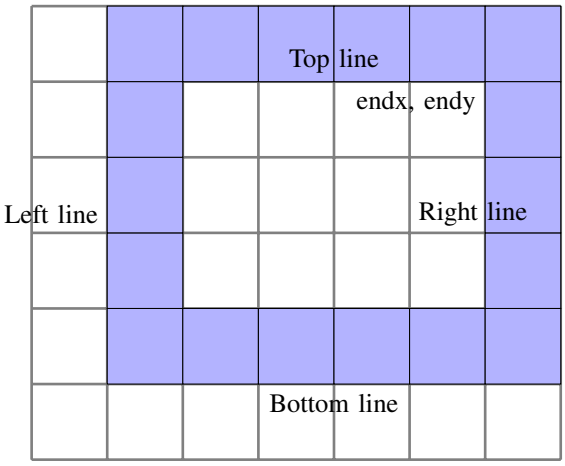
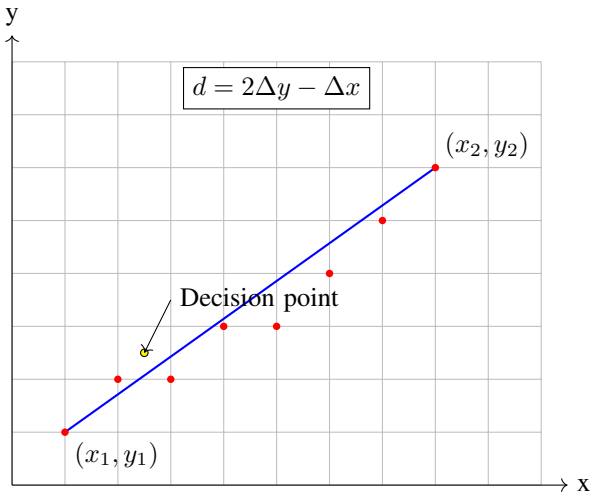


Fig. 9. This showcases the 1px wide box that would be partially drawn on the screen. The memory space concerning the center of the box is not written to at all

- **GLCD_DrawLine:** Bresenham's line algorithm implementation:
  - Handles all line orientations efficiently
  - Supports steep and shallow line slopes
  - Includes optimizations for straight lines

H. Visual Representation



If $d \leq 0$: Move East (E)
If $d > 0$: Move Northeast (NE)

Update d: $d_{new} = \begin{cases} d + 2\Delta y & \text{for E} \\ d + 2(\Delta y - \Delta x) & \text{for NE} \end{cases}$

Fig. 10. This showcases how the line algorithm works

Implementation Details

I. Initial Setup

The algorithm begins with several key preprocessing steps:

1) **Steep Line Detection:**
   - Determines if $|y_2 - y_1| > |x_2 - x_1|$
   - If true, transposes x and y coordinates for better precision
2) **Left-to-Right Ordering:**
   - Ensures $x_1 \leq x_2$ by swapping endpoints if necessary
   - Maintains consistent drawing direction

Core Variables The algorithm uses the following key variables:

- $dx = x_2 - x_1$ (Change in x)
- $dy = |y_2 - y_1|$ (Absolute change in y)
- $d = 2dy - dx$ (Decision parameter)
- $incrE = 2dy$ (East increment)
- $incrNE = 2(dy - dx)$ (Northeast increment)

Drawing Process For each x position from x_1 to x_2:

1) Draw pixel at current (x,y)
2) If $d \leq 0$:
   - Move East: x increases, y stays same
   - Update $d = d + incrE$
3) If $d > 0$:
   - Move Northeast: x increases, y changes by stepY
   - Update $d = d + incrNE$
4) Handle steep vs. non-steep cases when placing pixels

Optimizations The implementation includes several optimizations:

- Uses integer arithmetic throughout for better performance
- Handles both positive and negative slopes via stepY variable
- Properly transposes coordinates for steep lines to maintain accuracy
- Minimizes floating-point operations

Function Interface

```
void GLCD_DrawLine
(int x1, int y1, int x2, int y2, \\
unsigned short color)
```

Parameters:

- (x1, y1): Starting point coordinates
- (x2, y2): Ending point coordinates
- color: Color value for the line

- **GLCD_DrawCircle and GLCD_DrawFilledCircle:** Circle rendering:
  - Uses optimized Bresenham's circle algorithm
  - Implements efficient circle filling using horizontal line scanning
  - Includes octant symmetry optimizations

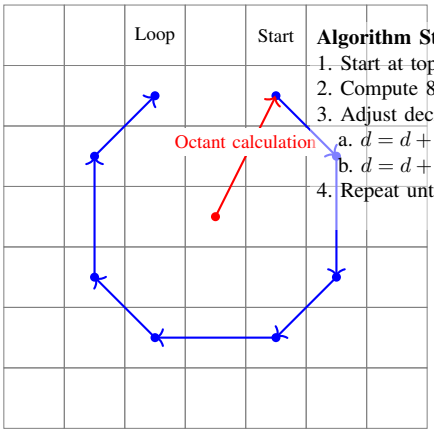


Fig. 11. Circle Drawing Algorithm

This is a figure which showcases how the algorithm works. You provide a radius and a center point and it draws the 8 pixels at appropriate areas and then it rotates and draws 8 more, that continues until the entire circle is filled. This is an efficient way to draw circles without the usage of bitmaps.

1) Transparency and Text Rendering: New functionality was added to support advanced rendering features:

- **GLCD_BitmapCutout:** Transparency support:
  - Implements alpha color keying
  - Optimizes transparent pixel skipping
  - Maintains background preservation

  Every new custom graphic function that I created included a bunch of fixes and improvements to the ones provided to us including:
  - **Bounds Checking**: The modified function includes more comprehensive bounds checking to handle cases where the bitmap is partially or completely outside the display area. This includes checking for cases where the bitmap is completely out of bounds, as well as handling x-axis and y-axis overflows and underflows. The modified function introduces logic to clip the bitmap to the display's vertical bounds, ensuring that only the visible portion of the bitmap is drawn.
  - **Error Handling**: The modified function returns error codes (-1 for invalid parameters, -2 for completely out of bounds) to allow the caller to handle issues more gracefully, rather than just drawing the bitmap regardless of the input parameters.
  - **Improved Drawing Logic**: The modified function calculates the actual drawing dimensions based on the bitmap size and the display's bounds, and only loops through the visible rows and columns of the bitmap. This is more efficient than the original function, which always looped through the entire bitmap regardless of visibility.
  - **Removal of Bitmap Flipping**: The original function required the bitmap data to be flipped vertically, which added complexity for the caller. The modified function removes this requirement, simplifying the usage of the function.

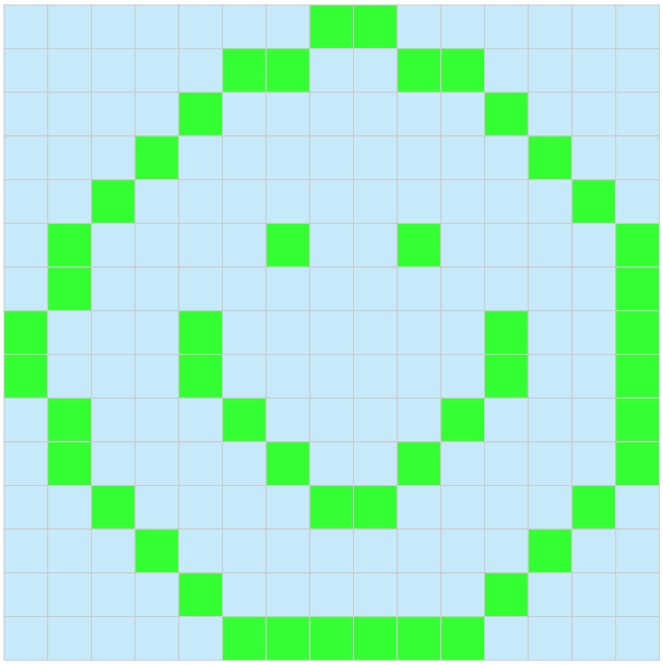


Fig. 12. Smiley Face Diagram. The green pixels represent any bitmap color data while the light blue pixels represent the specified alpha keying color which will be ignored by the function and skipped over to create a cutout of the provided bitmap

- **GLCD_DrawCharCutout:** Enhanced text rendering:
  - Supports background preservation
  - Optimizes character rendering for different font sizes
  - Implements efficient pixel-level control

Each character of the two provided fonts are bitmaps as well. The bitmaps are white for where pixels where the letters are supposed to be and black for where the pixels where they should not appear. Therefore using the same methodology as I used to draw the cutout bitmaps for the icons on the main menu, I was able to enable transparent lettering. This is useful for UI applications or for games where it is useful to write letters on top of other graphics without destroying the underlying background. I used this method to draw my name and the title of the project on the main menu without the need for an additional bitmap containing my name and title or even the need to refresh a large section of the screen. By default, to print a string you need to set a character color and a background color and the x,y coordinates and then the standard string drawing function will completely overwrite the memory space of the LCD driver board corresponding to the area specified, background and foreground included. My function simply checks the character font bitmap and ignores black pixels, sending dummy read commands for those and only sending write commands to the SPI interface for the font pixels that are white. With my custom function I could even change the color of the font written, instead of sending the command to draw a white pixel where the letter was expected

I was able to pass in a parameter which changed the color of the font. These features, transparancy, font coloring, and font size selection enabled me to create a diverse set of user interfaces for the game start menu, the gallery, the main menu, the boot up sequence and the mp3 player.

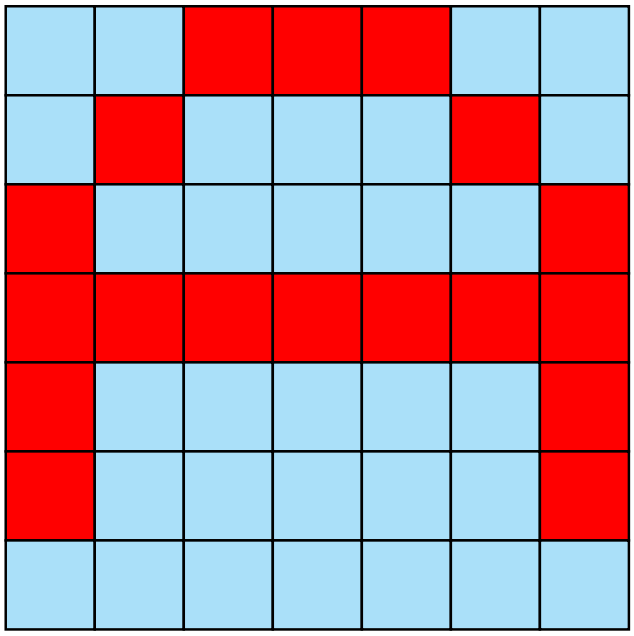


Fig. 13. Transparent Lettering Method where blue pixels can be ignored while only red ones indicate the presence of lettering

2) Screen Transitions: A comprehensive transition system was implemented to enhance user experience:

- Multiple transition patterns supported:
  - Left-to-right and right-to-left
  - Top-to-bottom and bottom-to-top
  - Center-out and outside-in
- Optimization techniques:
  - Box-based rendering for efficient updates
  - Pre-calculated transition coordinates
  - Manhattan distance calculations for radial patterns
  - No bitmaps were used to create smooth transitions without memory reads

The ‘transition_screen()‘ function is responsible for animating a transition effect by filling the screen with boxes in a specified pattern. The function takes three parameters:

- `color`: The color to be used for the boxes.
- `pattern`: An integer value representing the desired transition pattern.
- `delay_ms`: The delay in milliseconds between the drawing of each box.

The function first calculates the number of boxes that can fit on the screen, based on the screen dimensions and the box size (defined as `BOX_SIZE`). It then creates an array of `BoxCoord` structures to hold the coordinates of each box.

The function supports the following transition patterns:

1) **PATTERN_LEFT_TO_RIGHT**: The boxes are drawn from left to right, row by row.
2) **PATTERN_RIGHT_TO_LEFT**: The boxes are drawn from right to left, row by row.
3) **PATTERN_TOP_TO_BOTTOM**: The boxes are drawn from top to bottom, column by column.
4) **PATTERN_BOTTOM_TO_TOP**: The boxes are drawn from bottom to top, column by column.
5) **PATTERN_CENTER_OUT**: The boxes are drawn from the center of the screen outwards, based on their Manhattan distance from the center.
6) **PATTERN_OUTSIDE_IN**: The boxes are drawn from the edges of the screen inwards, based on their Manhattan distance from the center.

The function first pre-calculates the order in which the boxes should be drawn, based on the selected pattern. For the `PATTERN_CENTER_OUT` and `PATTERN_OUTSIDE_IN` patterns, the function uses a bubble sort algorithm to sort the boxes by their Manhattan distance from the center of the screen.

Once the box order has been determined, the function loops through the boxes and calls the `GLCD_FillBox()` function to draw each box. If a `delay_ms` value is provided, the function will pause for the specified duration between drawing each box, creating the desired transition effect.

The key steps of the ‘transition_screen()‘ function are:

1) Calculate the number of boxes that can fit on the screen.
2) Create an array of `BoxCoord` structures to hold the box coordinates.
3) Determine the box drawing order based on the selected pattern.
4) Loop through the boxes and call `GLCD_FillBox()` to draw each box.
5) If a `delay_ms` value is provided, pause for the specified duration between drawing each box.

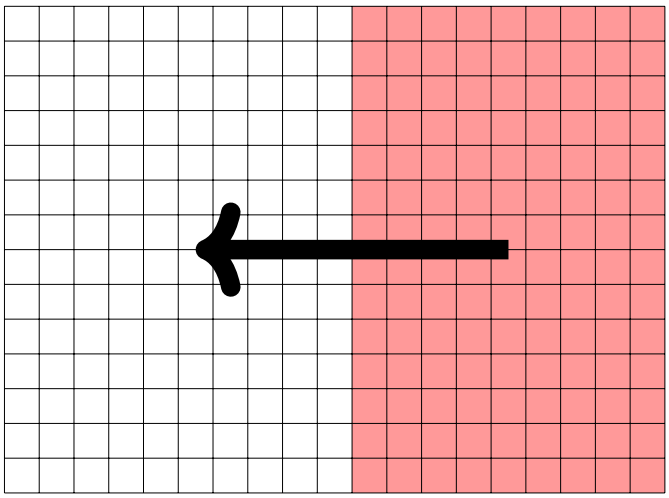


Fig. 14. Right-to-Left Transition Pattern using 16x16 boxes on the 320x240 display

The transition patterns provided by this function allow for a variety of visually appealing effects when updating the screen, such as sliding, expanding from the center, or a wave of color from the border to the center of the screen.

To conclude this section on graphics, the enhanced graphics library significantly improved the visual quality and performance of the media center, enabling smooth animations in the game component and responsive user interface transitions. Memory usage was optimized through careful bounds checking and partial updates, while maintaining visual fidelity and responsiveness.

V. Experimental Results

A. Performance Analysis

Testing revealed the various optimizations that I created paid off, as the game was able to run smoothly and every operation on the board was able to happen with minimal screen tearing. Having a realtime top down survival game was an especially difficult test of performance as there could be dozens of coins on the screen, dozens of upgrades, enemies and obstacles all at once which needed to be refreshed each tick. The ticks per second also had to surpass a minimum threshold in order to maintain the illusion of motion and ensure that the player can visualize the motion of all the enemies. Overall the game implementation was a success with the caveat that the idea of projectiles had to be abandoned because the game was already approaching its graphical limitations and putting more draw calls would likely make the game feel sluggish.

B. Memory Usage

Resource utilization measurements showed:

- Total Usage: 493204 bytes (96% utilization)

C. System Stability

Stress testing demonstrated:

- Stable operation under maximum load in the game with 32 enemies present
- Successful state transitions, the mp3 player could mute and unmute correctly when entering and leaving its subprocess
- Reliable peripheral interaction, the speaker would connect on demand and play audio from the computer

D. Difficulties

The audio and the game were the two most difficult parts to get right. In the end although my audio player worked and could initialize and mute and unmute, one issue I encountered was getting the audio quality to improve. After the demo, the TA told me that the audio quality was poor due to a mismatch between the core clock of the board and the audio interrupts. I looked into this and realized I had altered the core clock of the board in a previous step in my endeavor to increase its performance which was what was causing the audio fidelity to decrease.

The game was also difficult to implement as it relied on all the previous graphical library functions that I developed to work correctly and performantly. One issue that I had to fix was that the transparency draw calls were not working. I had previously used a more elaborate buffering mechanism to figure out a way for the draw call to skip large batches of unwanted sections of bitmaps all at once and it would cause an issue where the bitmap draws would be completely corrupted. I managed to redo the code and instead of using a buffering mechanism I individually skipped unwanted pixels by first checking if they matched the alpha_color that was passed in as a function parameter and if it did, then it would end the write operation with wr_dat_stop() and subsequently send the dummy read operation (void)rd_dat().

There was an issue with threading where the USB audio drivers would interfere any time threads were present and I tried diagnosing the issue and found a potential fix for it online by clearing the FRAME_INT interrupt flag at line 701 in the usbhw.c however, this was late in development and so I decided to use a state machine and disable my timer which would update the input state because it was easier to diagnose issues and it was too late to switch back to using a threading mechanism.

VI. Conclusions

The Media Center Project successfully demonstrates the integration of multiple embedded systems concepts:

- Efficient peripheral interfacing
- Modular software architecture
- Memory-optimized graphics rendering

Future improvements could include:

- Dynamic scheduling implementation
- Enhanced graphics capabilities
- Additional game features
- Extended multimedia format support

The project achieves its objectives while providing insights into embedded systems development challenges and solutions.

The game in particular turned out better than I had expected. I am proud to have completed such a complex game which featured realtime enemy AI and inputs. Most other people I saw created more basic games and did not enhance the graphical capabilities of the board which meant they were stuck drawing things to a low resolution grid of the screen meant for fonts. This game took considerable time to develop so I didn't have the time to create another one for the bonus points. In the end I wish I had time to diagnose the audio quality issue especially knowing now that it would have been an easy fix. Some more development time would have also allwoed me to implement a more extensible threading mechanism which would take more advantage of RTX threading.

REFERENCES

[1] DisplayFuture, "HX8347-D TFT LCD Controller Datasheet," Accessed: 2024-11-19. [Online]. Available: https://www.displayfuture.com/engineering/specs/controller/HX8347-D.pdf

[2] Keil, "MCB1700 User's Manual," Accessed: 2024-11-19. [Online]. Available: https://www.keil.com/support/man/docs/mcb1700/default.htm

[3] NXP Semiconductors, "LPC1769/68/67/66/65/64/63: 32-bit ARM Cortex-M3 Microcontroller Datasheet," Accessed: 2024-11-19. [Online].

[4] Toronto Metropolitan University, Department of Computer Engineering, "Media Center Lab Manual," Accessed: 2024-11-19. [Online].

APPENDIX

A. *The following are my custom graphical functions:*

```
/*************************************************************************
// Draws characters to screen without effecting background
*************************************************************************/
void GLCD_DrawCharCutout(unsigned int x, unsigned int y, unsigned int cw, unsigned int ch, unsigned char *c
    , unsigned short color) {
    unsigned int i, j, k, pixs;
    int wr_started = 0;

    GLCD_SetWindow(x, y, cw, ch);
    wr_cmd(0x22);

    k = (cw + 7) / 8;

    if (k == 1) {
        for (j = 0; j < ch; j++) {
            pixs = *(unsigned char *)c;
            c += 1;

            for (i = 0; i < cw; i++) {
                if (pixs & (1 << i)) { // ((pixs >> i) & 1)
                    if (!wr_started) {
                        wr_dat_start();
                        wr_started = 1;
                    }
                    wr_dat_only(color);
                } else {
                    if (wr_started) {
                        wr_dat_stop();
                        wr_started = 0;
                    }
                    (void)rd_dat();
                }
            }
        }
    } else if (k == 2) {
        for (j = 0; j < ch; j++) {
            pixs = *(unsigned short *)c;
            c += 2;

            for (i = 0; i < cw; i++) {
                if (pixs & (1 << i)) {  // ((pixs >> i) & 1)
                    if (!wr_started) {
                        wr_dat_start();
                        wr_started = 1;
                    }
                    wr_dat_only(color);
                } else {
                    if (wr_started) {
                        wr_dat_stop();
                        wr_started = 0;
                    }
                    (void)rd_dat();
                }
            }
        }
    }

    if (wr_started) {
        wr_dat_stop();
    }
}
void GLCD_DisplayCharCutout (unsigned int ln, unsigned int col, unsigned char fi, unsigned char c, unsigned
     short color) {

  c -= 32;
  switch (fi) {
    case 0:  /* Font 6 x 8 */
      GLCD_DrawCharCutout(col *  6, ln *  8,  6,  8, (unsigned char *)&Font_6x8_h  [c * 8], color);
      break;
    case 1:  /* Font 16 x 24 */
```

```
      GLCD_DrawCharCutout(col * 16, ln * 24, 16, 24, (unsigned char *)&Font_16x24_h[c * 24], color);
      break;
  }
}
void GLCD_DisplayStringCutout (unsigned int ln, unsigned int col, unsigned char fi, unsigned char *s,
    unsigned short color) {

  while (*s) {
    GLCD_DisplayCharCutout(ln, col++, fi, *s++, color);
  }
}


/*********************************************************************************
* Display graphical bitmap image at position x horizontally and y vertically   *
* with improved bounds checking, y-axis clipping, and error handling           *
* (This function is optimized for 16 bits per pixel format, it has to be       *
*  adapted for any other bits per pixel format)                                *
*   Parameter:      x:        horizontal position                              *
*                   y:        vertical position                                *
*                   w:        width of bitmap                                  *
*                   h:        height of bitmap                                 *
*                   bitmap:   address at which the bitmap data resides         *
*   Return:                                                                    *
* Modified by Victor Do (Images also do not need to be flipped anymore)        *
*********************************************************************************/
int GLCD_Bitmap(int x, int y, unsigned int w, unsigned int h, const unsigned char *bitmap) {
    int i, j;
    int startx = 0, endx = w;
    int starty = 0, endy = h;
    const unsigned short *bitmap_ptr = (const unsigned short *)bitmap;
    unsigned int draw_width = endx - startx;
    unsigned int draw_height = endy - starty;

          // Parameter validation
    if (!bitmap || w == 0 || h == 0) {
        return -1;  // Invalid parameters
    }

    // Complete out of bounds check
    if (x >= WIDTH || x + (signed int)w < 0 ||
        y >= HEIGHT || y + (signed int)h < 0) {
        return -2;  // Completely out of bounds
    }

    // X-axis bounds handling
    if (x + (signed int)w > WIDTH) {  // x overflow
        endx = WIDTH - x;
    } else if (x < 0) {  // x underflow
        startx = -x;
        x = 0;
    }

    // Y-axis bounds handling (new)
    if (y + (signed int)h > HEIGHT) {  // y overflow
        endy = HEIGHT - y;
    } else if (y < 0) {  // y underflow
        starty = -y;
        y = 0;
    }

          // Calculate actual drawing dimensions
    draw_width = endx - startx;
    draw_height = endy - starty;

    // Set the window for the visible portion only
    GLCD_SetWindow(x, y, draw_width, draw_height);

    // Start drawing
    wr_cmd(0x22);
    wr_dat_start();

    // Only loop through the visible rows
    for (i = starty * w; i < endy * w; i += w) {
```

```
        // Only draw visible pixels in each row
        for (j = startx; j < endx; j++) {
            wr_dat_only(bitmap_ptr[i + j]);
        }
    }
    wr_dat_stop();

    return 0;  // Success
}
/*************************************************************************
* Display portion of a graphical bitmap image within specified region    *
* This function allows redrawing only a specific section of the screen   *
* (This function is optimized for 16 bits per pixel format)              *
*   Parameters:     x:        horizontal position of bitmap              *
*                   y:        vertical position of bitmap                *
*                   w:        width of entire bitmap                     *
*                   h:        height of entire bitmap                    *
*                   bitmap:   address at which the bitmap data resides   *
*                   rx:       x position of region to redraw             *
*                   ry:       y position of region to redraw             *
*                   rw:       width of region to redraw                  *
*                   rh:       height of region to redraw                 *
*   Return:         0 on success, negative value on error                *
*************************************************************************/
int GLCD_Bitmap_Region(int x, int y, unsigned int w, unsigned int h,
                       const unsigned char *bitmap,
                       int rx, int ry, unsigned int rw, unsigned int rh) {
    int i, j;
    const unsigned short *bitmap_ptr = (const unsigned short *)bitmap;



    // Calculate intersection between bitmap and region
    int region_start_x = rx;
    int region_start_y = ry;
    int region_end_x = rx + rw;
    int region_end_y = ry + rh;

                // Calculate relative positions within bitmap
    int bitmap_start_x = region_start_x - x;
    int bitmap_start_y = region_start_y - y;
    int draw_width = region_end_x - region_start_x;
    int draw_height = region_end_y - region_start_y;

                // Parameter validation
    if (!bitmap || w == 0 || h == 0 || rw == 0 || rh == 0) {
        return -1;  // Invalid parameters
    }

    // Clip region to bitmap boundaries
    if (region_start_x < x) region_start_x = x;
    if (region_start_y < y) region_start_y = y;
    if (region_end_x > x + w) region_end_x = x + w;
    if (region_end_y > y + h) region_end_y = y + h;

    // Check if region is completely outside bitmap
    if (region_start_x >= region_end_x || region_start_y >= region_end_y) {
        return -2;  // No intersection
    }



    // Screen bounds checking
    if (region_start_x >= WIDTH || region_end_x <= 0 ||
        region_start_y >= HEIGHT || region_end_y <= 0) {
        return -3;  // Region completely out of screen bounds
    }

    // Clip to screen boundaries
    if (region_start_x < 0) {
        bitmap_start_x -= region_start_x;
        draw_width += region_start_x;
        region_start_x = 0;
    }
```

```
    if (region_end_x > WIDTH) {
        draw_width -= (region_end_x - WIDTH);
    }
    if (region_start_y < 0) {
        bitmap_start_y -= region_start_y;
        draw_height += region_start_y;
        region_start_y = 0;
    }
    if (region_end_y > HEIGHT) {
        draw_height -= (region_end_y - HEIGHT);
    }

    // Set the window for the visible portion only
    GLCD_SetWindow(region_start_x, region_start_y, draw_width, draw_height);

    // Start drawing
    wr_cmd(0x22);
    wr_dat_start();

    // Only loop through the visible rows of the region
    for (i = bitmap_start_y; i < bitmap_start_y + draw_height; i++) {
        // Calculate the starting position for this row in the bitmap
        int row_offset = i * w;
        // Only draw visible pixels in each row
        for (j = bitmap_start_x; j < bitmap_start_x + draw_width; j++) {
            wr_dat_only(bitmap_ptr[row_offset + j]);
        }
    }

    wr_dat_stop();
    return 0;  // Success
}

/*********************************************************************************
// Draws bitmap with cutout
*********************************************************************************/
int GLCD_BitmapCutout(int x, int y, unsigned int w, unsigned int h,
                      const unsigned char *bitmap, unsigned short alpha_color) {
    const unsigned short *bitmap_ptr = (const unsigned short *)bitmap;
    int startx = 0, endx = w;
    int starty = 0, endy = h;
    int wr_started = 0;
    unsigned int draw_width, draw_height;

    if (!bitmap || w == 0 || h == 0) return -1;
    if (x >= WIDTH || x + (signed int)w < 0 ||
        y >= HEIGHT || y + (signed int)h < 0) return -2;

    // X-axis bounds handling
    if (x + (signed int)w > WIDTH) {
        endx = WIDTH - x;
    } else if (x < 0) {
        startx = -x;
        x = 0;
    }

    // Y-axis bounds handling
    if (y + (signed int)h > HEIGHT) {
        endy = HEIGHT - y;
    } else if (y < 0) {
        starty = -y;
        y = 0;
    }

    draw_width = endx - startx;
    draw_height = endy - starty;

    GLCD_SetWindow(x, y, draw_width, draw_height);
    wr_cmd(0x22);

    // Optimized drawing loop
    for (int i = starty * w; i < endy * w; i += w) {
        for (int j = startx; j < endx; j++) {
            unsigned short current_pixel = bitmap_ptr[i + j];
```

```
290
291             if (current_pixel != alpha_color) {
292                 if (!wr_started) {
293                     wr_dat_start();
294                     wr_started = 1;
295                 }
296                 wr_dat_only(current_pixel);
297             } else {
298                 if (wr_started) {
299                     wr_dat_stop();
300                     wr_started = 0;
301                 }
302                 (void)rd_dat();
303             }
304         }
305     }
306
307     if (wr_started) {
308         wr_dat_stop();
309     }
310
311     return 0;
312 }
313
314 /*****************************************************************************
315 //This draws the pixel at coords with specified color
316 *****************************************************************************/
317 void GLCD_PutPixelc (int x, int y, unsigned short color) {
318
319         // Complete out of bounds check
320         if (x >= WIDTH || y >= HEIGHT || x < 0 || y < 0) {
321                         return;  // Completely out of bounds
322         }
323
324   if (Himax) {
325     wr_reg(0x02, x >>    8);            /* Column address start MSB           */
326     wr_reg(0x03, x &  0xFF);            /* Column address start LSB           */
327     wr_reg(0x04, x >>    8);            /* Column address end MSB             */
328     wr_reg(0x05, x &  0xFF);            /* Column address end LSB             */
329
330     wr_reg(0x06, y >>    8);            /* Row address start MSB              */
331     wr_reg(0x07, y &  0xFF);            /* Row address start LSB              */
332     wr_reg(0x08, y >>    8);            /* Row address end MSB                */
333     wr_reg(0x09, y &  0xFF);            /* Row address end LSB                */
334   }
335   else {
336    #if (LANDSCAPE == 1)
337     wr_reg(0x20, y);
338     wr_reg(0x21, x);
339    #else
340     wr_reg(0x20, x);
341     wr_reg(0x21, y);
342    #endif
343   }
344
345   wr_cmd(0x22);
346   wr_dat(color);
347 }
348
349 /*****************************************************************************
350 * Draw a 1-pixel thick outline of a box at position (x, y)                   *
351 * by: Victor Do                                                              *
352 * Parameters:                                                                *
353 *   x:      horizontal position                                              *
354 *   y:      vertical position       (starts at top left)                     *
355 *   w:      width of the outline box                                         *
356 *   h:      height of the outline box                                        *
357 *   color:  color value for the outline                                      *
358 * Return:                                                                    *
359 *****************************************************************************/
360 int GLCD_DrawBox(int x, int y, unsigned int w, unsigned int h, unsigned short color) {
361     int i;
362
363                 // Clip coordinates and dimensions
```

```
    int startx = (x < 0) ? 0 : x;
    int starty = (y < 0) ? 0 : y;
    int endx = (x + w > WIDTH) ? WIDTH : x + w;
    int endy = (y + h > HEIGHT) ? HEIGHT : y + h;

          // Recalculate dimensions after clipping
    unsigned int clipped_w = endx - startx;
    unsigned int clipped_h = endy - starty;

          // Complete out of bounds check
    if (x >= WIDTH || x + (signed int)w <= 0 ||
        y >= HEIGHT || y + (signed int)h <= 0) {
        return -1;  // Completely out of bounds
    }

    // Optimize for thin boxes
    if (clipped_w == 1) {
        // Single vertical line
        GLCD_SetWindow(startx, starty, 1, clipped_h);
        wr_cmd(0x22);
        wr_dat_start();
        for (i = 0; i < clipped_h; i++) {
            wr_dat_only(color);
        }
        wr_dat_stop();
        return 0;
    }

    if (clipped_h == 1) {
        // Single horizontal line
        GLCD_SetWindow(startx, starty, clipped_w, 1);
        wr_cmd(0x22);
        wr_dat_start();
        for (i = 0; i < clipped_w; i++) {
            wr_dat_only(color);
        }
        wr_dat_stop();
        return 0;
    }

                // Draw horizontal lines
    if (starty < endy) {
        // Top line
        GLCD_SetWindow(startx, starty, clipped_w, 1);
        wr_cmd(0x22);
        wr_dat_start();
        for (i = 0; i < clipped_w; i++) {
            wr_dat_only(color);
        }
        wr_dat_stop();

        if (clipped_h > 1) {
            // Bottom line
            GLCD_SetWindow(startx, endy - 1, clipped_w, 1);
            wr_cmd(0x22);
            wr_dat_start();
            for (i = 0; i < clipped_w; i++) {
                wr_dat_only(color);
            }
            wr_dat_stop();
        }
    }

    // Draw vertical lines (if height > 2)
    if (clipped_h > 2 && startx < endx) {
        // Left line
        GLCD_SetWindow(startx, starty + 1, 1, clipped_h - 2);
        wr_cmd(0x22);
        wr_dat_start();
        for (i = 0; i < clipped_h - 2; i++) {
            wr_dat_only(color);
        }
        wr_dat_stop();

```

```
        if (clipped_w > 1) {
            // Right line
            GLCD_SetWindow(endx - 1, starty + 1, 1, clipped_h - 2);
            wr_cmd(0x22);
            wr_dat_start();
            for (i = 0; i < clipped_h - 2; i++) {
                wr_dat_only(color);
            }
            wr_dat_stop();
        }
    }

    return 0;  // Success
}

/*********************************************************************************
* Draw a line using Bresenham's line algorithm                                   *
* by: Victor Do                                                                  *
*   Parameter:      x1, y1:   starting point of line                             *
*                   x2, y2:   ending point of line                               *
*                   color:  color value for the outline                          *
*   Return:                                                                      *
*********************************************************************************/
void GLCD_DrawLine(int x1, int y1, int x2, int y2, unsigned short color) {
    int dx, dy, incrE, incrNE, d, x, y;
    int steep = abs(y2 - y1) > abs(x2 - x1);
                int stepY;

    // If line is steep, transpose the coordinates
    if (steep) {
        int temp;
        temp = x1; x1 = y1; y1 = temp;
        temp = x2; x2 = y2; y2 = temp;
    }

    // Make sure we always draw from left to right
    if (x1 > x2) {
        int temp;
        temp = x1; x1 = x2; x2 = temp;
        temp = y1; y1 = y2; y2 = temp;
    }

    dx = x2 - x1;
    dy = abs(y2 - y1);
    d = (2 * dy) - dx;
    incrE = 2 * dy;
    incrNE = 2 * (dy - dx);

    y = y1;
    //int stepY = (y1 < y2) ? 1 : -1;
                if (y1 < y2){
                        stepY = 1;
                } else {
                        stepY = -1;
                }

    for (x = x1; x <= x2; x++) {
        if (steep) {
            GLCD_PutPixelc(y, x, color);
        } else {
            GLCD_PutPixelc(x, y, color);
        }

        if (d <= 0) {
            d += incrE;
        } else {
            y += stepY;
            d += incrNE;
        }
    }
}

/*********************************************************************************
* Draw a circle using Bresenham's circle algorithm                               *
```

```
* by: Victor Do                                                              *
*   Parameter:      xc, yc:   center point of circle                        *
*                   radius:   radius of circle                              *
*                   color:  color value for the outline                     *
*   Return:                                                                 *
*****************************************************************************/
#define SCREEN_WIDTH 320
#define SCREEN_HEIGHT 240
// Check if a pixel is within screen boundaries
int isPixelOnScreen(int x, int y) {
    return (x >= 0 && x < SCREEN_WIDTH && y >= 0 && y < SCREEN_HEIGHT);
}
// Plot initial points in all octants
void plotCirclePoints(int xc, int yc, int x, int y, unsigned short color) {
                // 8 symmetric points of the circle
    int points[8][2];
                int i = 0;

          // Populate points array
    points[0][0] = xc + x; points[0][1] = yc + y;
    points[1][0] = xc - x; points[1][1] = yc + y;
    points[2][0] = xc + x; points[2][1] = yc - y;
    points[3][0] = xc - x; points[3][1] = yc - y;
    points[4][0] = xc + y; points[4][1] = yc + x;
    points[5][0] = xc - y; points[5][1] = yc + x;
    points[6][0] = xc + y; points[6][1] = yc - x;
    points[7][0] = xc - y; points[7][1] = yc - x;

    // Plot each point if it's on screen
    for ( i = 0; i < 8; i++) {
        if (points[i][0] >= 0 && points[i][0] < SCREEN_WIDTH &&
            points[i][1] >= 0 && points[i][1] < SCREEN_HEIGHT) {
            GLCD_PutPixelc(points[i][0], points[i][1], color);
        }
    }
}

void GLCD_DrawCircle(int xc, int yc, int radius, unsigned short color) {
    int x = 0;
    int y = radius;
    int d = 3 - 2 * radius;

    // Quick reject if circle is completely off screen
    if (xc + radius < 0 || xc - radius >= SCREEN_WIDTH ||
        yc + radius < 0 || yc - radius >= SCREEN_HEIGHT) {
        return;  // Circle is entirely off screen
    }

    // Draw initial points
    plotCirclePoints(xc, yc, x, y, color);

    while (y >= x) {
        x++;

        // Update decision parameter
        if (d > 0) {
            y--;
            d = d + 4 * (x - y) + 10;
        } else {
            d = d + 4 * x + 6;
        }

        // Draw points in all octants
        plotCirclePoints(xc, yc, x, y, color);
    }
}

/*****************************************************************************
* Draw a filled circle using an optimized algorithm                          *
* by: Victor Do                                                              *
*   Parameter:      xc, yc:   center point of circle                        *
*                   radius:   radius of circle                              *
*                   color:  color value for the outline                     *
*   Return:                                                                 *
```

```
586 ********************************************************************************/
587 // Function to draw horizontal lines for filling
588 void drawHorizontalLine(int x1, int x2, int y, unsigned short color) {
589                 int i;
590                 GLCD_SetWindow(x1, y, x2 - x1 + 1, 1); // Set window for a horizontal line
591                 wr_cmd(0x22);
592                 wr_dat_start();
593                 for (i = x1; i <= x2; i++) {
594                                 wr_dat_only(color); // Assuming 0xFFFF is the color value
595                 }
596                 wr_dat_stop();
597 }
598
599 void GLCD_DrawFilledCircle(int xc, int yc, int radius, unsigned short color) {
600     int x = 0;
601     int y = radius;
602     int d = 3 - 2 * radius;
603
604     // Draw initial points and lines
605     while (y >= x) {
606         // Draw horizontal lines to fill the circle
607         drawHorizontalLine(xc - x, xc + x, yc + y, color);
608         drawHorizontalLine(xc - x, xc + x, yc - y, color);
609         drawHorizontalLine(xc - y, xc + y, yc + x, color);
610         drawHorizontalLine(xc - y, xc + y, yc - x, color);
611
612         x++;
613
614         if (d > 0) {
615             y--;
616             d = d + 4 * (x - y) + 10;
617         } else {
618             d = d + 4 * x + 6;
619         }
620     }
621 }
622
623 /*******************************************************************************
624 * Fill a rectangular box with specified position, dimensions and color          *
625 * Includes complete bounds checking and optimized filling                       *
626 ********************************************************************************/
627 int GLCD_FillBox(int x, int y, unsigned int w, unsigned int h, unsigned short color) {
628           // Clip coordinates and dimensions
629     int startx = (x < 0) ? 0 : x;
630     int starty = (y < 0) ? 0 : y;
631     int endx = (x + w > WIDTH) ? WIDTH : x + w;
632     int endy = (y + h > HEIGHT) ? HEIGHT : y + h;
633                 unsigned int i;
634     unsigned int total_pixels;
635
636     // Recalculate dimensions after clipping
637     unsigned int clipped_w = endx - startx;
638     unsigned int clipped_h = endy - starty;
639
640             // Complete out of bounds check
641     if (x >= WIDTH || x + (signed int)w <= 0 ||
642         y >= HEIGHT || y + (signed int)h <= 0) {
643         return -2;  // Completely out of bounds
644     }
645
646                 // Set window once for the entire fill operation
647     GLCD_SetWindow(startx, starty, clipped_w, clipped_h);
648     wr_cmd(0x22);
649     wr_dat_start();
650
651     // Calculate total number of pixels
652     total_pixels = clipped_w * clipped_h;
653
654     // Optimize the filling loop to avoid unnecessary row/column calculations
655     for (i = 0; i < total_pixels; i++) {
656         wr_dat_only(color);
657     }
658
659     wr_dat_stop();
```

```
    return 0;  // Success
}

/*******************************************************************************
//Transitions:
*******************************************************************************/
#define BOX_SIZE 16
#define BOXES_X (WIDTH / BOX_SIZE)  // 20 boxes
#define BOXES_Y (HEIGHT / BOX_SIZE) // 15 boxes

// Structure to hold box coordinates for transition ordering
typedef struct {
    int x;
    int y;
} BoxCoord;

// Calculate Manhattan distance from center
static int manhattan_dist(int x1, int y1, int x2, int y2) {
    return abs(x1 - x2) + abs(y1 - y2);
}

// Function to fill screen with boxes in specified pattern
void transition_screen(unsigned short color, int pattern, int delay_ms) {
    BoxCoord box_order[BOXES_X * BOXES_Y];
    int total_boxes = BOXES_X * BOXES_Y;
    int box_count = 0;
    int center_x = BOXES_X / 2;
    int center_y = BOXES_Y / 2;
    int x, y, i, j;
    int dist1, dist2;
    BoxCoord temp;

    // Pre-calculate box order based on pattern
    switch(pattern) {
        case PATTERN_LEFT_TO_RIGHT:
            x = 0;
            y = 0;
            for(x = 0; x < BOXES_X; x++) {
                for(y = 0; y < BOXES_Y; y++) {
                    box_order[box_count].x = x;
                    box_order[box_count].y = y;
                    box_count++;
                }
            }
            break;

        case PATTERN_RIGHT_TO_LEFT:
            x = BOXES_X - 1;
            y = 0;
            for(x = BOXES_X - 1; x >= 0; x--) {
                for(y = 0; y < BOXES_Y; y++) {
                    box_order[box_count].x = x;
                    box_order[box_count].y = y;
                    box_count++;
                }
            }
            break;

        case PATTERN_TOP_TO_BOTTOM:
            x = 0;
            y = 0;
            for(y = 0; y < BOXES_Y; y++) {
                for(x = 0; x < BOXES_X; x++) {
                    box_order[box_count].x = x;
                    box_order[box_count].y = y;
                    box_count++;
                }
            }
            break;

        case PATTERN_BOTTOM_TO_TOP:
            x = 0;
            y = BOXES_Y - 1;
            for(y = BOXES_Y - 1; y >= 0; y--) {
```

```
                for(x = 0; x < BOXES_X; x++) {
                    box_order[box_count].x = x;
                    box_order[box_count].y = y;
                    box_count++;
                }
            }
            break;

        case PATTERN_CENTER_OUT:
        case PATTERN_OUTSIDE_IN:
            x = 0;
            y = 0;
            // Create array of boxes
            for(y = 0; y < BOXES_Y; y++) {
                for(x = 0; x < BOXES_X; x++) {
                    box_order[box_count].x = x;
                    box_order[box_count].y = y;
                    box_count++;
                }
            }
            // Bubble sort boxes by distance from center
            for(i = 0; i < total_boxes - 1; i++) {
                for(j = 0; j < total_boxes - i - 1; j++) {
                    dist1 = manhattan_dist(box_order[j].x, box_order[j].y, center_x, center_y);
                    dist2 = manhattan_dist(box_order[j+1].x, box_order[j+1].y, center_x, center_y);
                    // For center out: sort ascending, for outside in: sort descending
                    if((pattern == PATTERN_CENTER_OUT && dist1 > dist2) ||
                       (pattern == PATTERN_OUTSIDE_IN && dist1 < dist2)) {
                        temp = box_order[j];
                        box_order[j] = box_order[j+1];
                        box_order[j+1] = temp;
                    }
                }
            }
            break;
    }

    // Draw boxes in calculated order
    i = 0;
    for(i = 0; i < total_boxes; i++) {
        GLCD_FillBox(
            box_order[i].x * BOX_SIZE,
            box_order[i].y * BOX_SIZE,
            BOX_SIZE,
            BOX_SIZE,
            color
        );

        if(delay_ms > 0) {
            //delay(delay_ms);  // Add delay between boxes if specified
        }
    }
}
```

B. The following is the code from my Main.c:

```
//Final Project
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <math.h>
#include <stdint.h>

#define osObjectsPublic                     // define objects in main module
#include "osObjects.h"                      // RTOS object definitions
#include "cmsis_os.h"                                       // CMSIS RTOS header file
#include "LPC17xx.h"
#include "GLCD.h"
#include "KBD.h"
#include "USBAudio/type.h"
#include "GameScript.h"
#include "MainMenu.h"

```

```c
#define __USE_LCD 1 /* Uncomment to use the LCD */


// Global variables and other shared resources Definition
char buf[20];
extern uint32_t KBD_val;
int kbdEnabled = 0;
uint64_t tickCount = 0;
uint64_t ticks = 0;
//Main Menu Definitions


// Gallery Viewer Definitions
#define gallerySize 4 //4
//extern unsigned char COLORFUL_BG_pixel_data[];
//extern unsigned char SWORD_pixel_data[];
//extern unsigned char BLUEFLOWER_pixel_data[];
#include "Images/colorful-BG.h"
#include "Images/sword.h"
#include "Images/blueFlower.h"
#include "Images/castle.h"
const unsigned char *gallery[gallerySize] = {COLORFUL_BG_pixel_data, SWORD_pixel_data,
    BLUEFLOWER_pixel_data, CASTLE_pixel_data};
//const unsigned char *gallery[gallerySize] = {BLUEFLOWER_pixel_data};

int currentImage = 0;
int prevImage = -1;

//USBAudio Definitions
extern int USBAudio_Init(void);
extern uint8_t Mute;

//State Machine Definitions
#define STATE_MAIN_MENU    0
#define STATE_GALLERY      1
#define STATE_MP3_PLAYER   2
#define STATE_GAME         3
#define TOTAL_STATES       4

typedef void (*InitializationHandler)(void);
typedef void (*UpdateHandler)(uint32_t ticks);
typedef void (*InputHandler)(uint32_t key);

typedef struct {
  InitializationHandler init, exit;
  UpdateHandler         update;
  InputHandler          input;
}  StateDef;

StateDef States[TOTAL_STATES];

int curr_state = STATE_MAIN_MENU;
int prev_state = -1;


//extern void Init_Threads     (void);
//extern void Terminate_Threads(void);
//extern osThreadId tid_MainMenu, tid_Gallery, tid_MP3Player, tid_Game, tid_UserInput;

extern void MainMenu_Init(void);

void Gallery_Start(){
        GLCD_Clear(Cyan);
        transition_screen(Black, rand() % 6,  0);               //Black Transition - Random Pattern
        currentImage = 0;
        prevImage = -1;
        //GLCD_Clear(Olive);
        GLCD_SetBackColor(Black);
        GLCD_SetTextColor(White);
        GLCD_DisplayStringCutout(0, 0, 1 , "␣␣␣Gallery␣Images␣␣␣", White);
        GLCD_DisplayStringCutout(9, 0, 1 , "␣␣␣␣Image␣1␣of␣4␣␣␣␣", White);
}

void Gallery_Update(uint32_t ticks){
```

```
        int posX, posY = 0;
        int sizeX, sizeY = 0;


        if(currentImage == 0)
        {
                posX = 20;
                posY = 20;
                sizeX = COLORFUL_BG_WIDTH;
                sizeY = COLORFUL_BG_HEIGHT;
                //GLCD_DisplayStringCutout(9, 0, 1 , "    Image 1 of 4    ", White);
                GLCD_DisplayString(9, 0, 1 , "____Image 1 of 4____");
        }
        else if(currentImage == 1)
        {
                posX = 124;
                posY = 76;
                sizeX = SWORD_WIDTH;
                sizeY = SWORD_HEIGHT;
                GLCD_DisplayStringCutout(9, 0, 1 , "____Image 2 of 4____", White);
                //GLCD_DisplayString(9, 0, 1 , "    Image 2 of 4    ");
        }
        else if(currentImage == 2)
        {
                posX = 135;
                posY = 4;
                sizeX = BLUEFLOWER_WIDTH;
                sizeY = BLUEFLOWER_HEIGHT;
                GLCD_DisplayStringCutout(9, 0, 1 , "____Image 3 of 4____", White);
                //GLCD_DisplayString(9, 0, 1 , "    Image 3 of 4    ");
        }
        else if(currentImage == 3)
        {
                posX = 0;
                posY = 20;
                sizeX = CASTLE_WIDTH;
                sizeY = CASTLE_HEIGHT;
                GLCD_DisplayStringCutout(9, 0, 1 , "____Image 4 of 4____", White);
                //GLCD_DisplayString(9, 0, 1 , "    Image 4 of 4    ");
        }

        if(prevImage != currentImage){
                transition_screen(0x0000, rand() % 6,  0);              //Black Transition - Random Pattern
                //GLCD_Clear(Black);
                GLCD_Bitmap(posX, posY, sizeX, sizeY, gallery[currentImage]);
                prevImage = currentImage;
        }
}

void MP3_Player_Start()
{
        USBAudio_Init();
        transition_screen(0x0000, rand() % 6,  0);              //Black Transition - Random Pattern
        //GLCD_Clear(Black);
        GLCD_SetBackColor(Black);
        GLCD_SetTextColor(White);
        GLCD_DisplayStringCutout(0, 0, 1 , "_____MP3 PLAYER_____", White);
        GLCD_DisplayString(24, 0, 0, "____________________HOLD UP TO EXIT____________________");
        GLCD_DisplayString(27, 0, 0, "________________Created by Victor Do________________");
        Mute = FALSE;
}

// Assuming these are defined elsewhere
extern uint32_t Volume;  // From usbdmain.c
#define SCREEN_WIDTH 320
#define SCREEN_HEIGHT 240
#define MAX_VOLUME 16   // Adjust based on your volume range
#define BASE_RADIUS 30   // Base radius for the circle
#define PULSE_AMPLITUDE 10  // How much the circle pulsates
#define PULSE_SPEED 0.1f   // Speed of pulsation
void MP3_Player_Update(uint32_t ticks)
{
        Mute = FALSE;

```

```
        // Calculate center coordinates
    int centerX = SCREEN_WIDTH / 2;
    int centerY = SCREEN_HEIGHT / 2;

    // Calculate base radius based on volume level
    int baseRadius = (int)(BASE_RADIUS * Volume);

    // Calculate pulsation using sine wave
    float pulseOffset = (sinf(ticks * PULSE_SPEED) + 2) * PULSE_AMPLITUDE;

    // Calculate final radius with pulsation
    int finalRadius = baseRadius + (int)pulseOffset;

    // Ensure radius stays positive
    if (finalRadius < 1) finalRadius = 1;

    // Clear previous frame (optional - depending on your display handling)
    GLCD_DrawCircle(centerX, centerY, finalRadius + 1, 0x0000); // Black

    // Draw new circle
    GLCD_DrawCircle(centerX, centerY, finalRadius, 0xFFFF); // White

}

void MP3_Player_Exit()
{
        Mute = TRUE;
}
void delay (int cnt) {
  cnt <<= 18;
  while (cnt--);
}
void GAME_Exit(){
        uint32_t joystick_state;
    transition_screen(Red, rand() % 6, PATTERN_TOP_TO_BOTTOM);
    GLCD_SetBackColor(Black);
    GLCD_SetTextColor(White);
    GLCD_DisplayString(4, 0, 1 , "      YOU DIED      ");
    GLCD_DisplayString(21, 0, 0, "                Better Luck Next Time                ");
    GLCD_DisplayString(24, 0, 0, "                Created by Victor Do                ");

    delay(50);

    while((joystick_state & KBD_SELECT) == 0) {

        joystick_state = get_button();
    }
}

void incrementTicks(){
        if (ticks++ >= 9) {                     /* Set Clock1s to 10ms */
                        ticks    = 0;
                        tickCount += 1;
                }
}

void draw_connected_lines(int x_offset, int y_offset, int clear) {
    static int phase = 0;
    int x_shift = phase % 2;  // Oscillates between 0 and 1
    int y_shift = (phase % 6) / 2;  // Cycles through 0,1,2

    phase = (phase + 1) % 6;  // Increment phase, wrap at 6

    int final_x = x_offset + x_shift;
    int final_y = y_offset + y_shift * 3;  // Multiply by 3 for larger vertical movement

    unsigned short color_main = clear ? Black : Red;
    unsigned short color_secondary = clear ? Black : White;

    #define OFFSET(x,y) (x + final_x), (y + final_y)

    // Upper section
    GLCD_DrawLine(OFFSET(66, 15), OFFSET(77, 62), color_main);
    GLCD_DrawLine(OFFSET(77, 62), OFFSET(89, 14), color_main);
```

```
    GLCD_DrawLine(OFFSET(93, 59), OFFSET(104, 15), color_main);
    GLCD_DrawLine(OFFSET(104, 15), OFFSET(114, 57), color_main);
    GLCD_DrawLine(OFFSET(98, 39), OFFSET(109, 36), color_main);
    GLCD_DrawLine(OFFSET(121, 56), OFFSET(118, 15), color_main);
    GLCD_DrawLine(OFFSET(118, 15), OFFSET(129, 32), color_main);
    GLCD_DrawLine(OFFSET(129, 32), OFFSET(133, 14), color_main);
    GLCD_DrawLine(OFFSET(133, 14), OFFSET(143, 53), color_main);
    GLCD_DrawLine(OFFSET(149, 52), OFFSET(146, 15), color_main);
    GLCD_DrawLine(OFFSET(146, 15), OFFSET(163, 16), color_main);
    GLCD_DrawLine(OFFSET(163, 16), OFFSET(147, 33), color_main);
    GLCD_DrawLine(OFFSET(171, 15), OFFSET(175, 47), color_main);
    GLCD_DrawLine(OFFSET(187, 49), OFFSET(183, 14), color_main);
    GLCD_DrawLine(OFFSET(183, 14), OFFSET(200, 22), color_main);
    GLCD_DrawLine(OFFSET(200, 22), OFFSET(187, 34), color_main);
    GLCD_DrawLine(OFFSET(187, 34), OFFSET(203, 46), color_main);
    GLCD_DrawLine(OFFSET(213, 14), OFFSET(216, 48), color_main);
    GLCD_DrawLine(OFFSET(216, 48), OFFSET(236, 46), color_main);
    GLCD_DrawLine(OFFSET(215, 34), OFFSET(229, 32), color_main);
    GLCD_DrawLine(OFFSET(213, 15), OFFSET(237, 12), color_main);

    // Lower section
    GLCD_DrawLine(OFFSET(109, 80), OFFSET(97, 71), color_main);
    GLCD_DrawLine(OFFSET(97, 71), OFFSET(87, 86), color_main);
    GLCD_DrawLine(OFFSET(87, 86), OFFSET(111, 95), color_main);
    GLCD_DrawLine(OFFSET(111, 95), OFFSET(102, 108), color_main);
    GLCD_DrawLine(OFFSET(102, 108), OFFSET(91, 100), color_main);
    GLCD_DrawLine(OFFSET(115, 75), OFFSET(119, 100), color_main);
    GLCD_DrawLine(OFFSET(119, 100), OFFSET(130, 97), color_main);
    GLCD_DrawLine(OFFSET(130, 97), OFFSET(126, 72), color_main);
    GLCD_DrawLine(OFFSET(138, 95), OFFSET(133, 69), color_main);
    GLCD_DrawLine(OFFSET(133, 69), OFFSET(144, 73), color_main);
    GLCD_DrawLine(OFFSET(143, 73), OFFSET(137, 85), color_main);
    GLCD_DrawLine(OFFSET(137, 85), OFFSET(151, 92), color_main);
    GLCD_DrawLine(OFFSET(153, 72), OFFSET(163, 92), color_main);
    GLCD_DrawLine(OFFSET(163, 92), OFFSET(166, 66), color_main);
    GLCD_DrawLine(OFFSET(170, 64), OFFSET(174, 89), color_main);
    GLCD_DrawLine(OFFSET(177, 66), OFFSET(186, 86), color_main);
    GLCD_DrawLine(OFFSET(190, 64), OFFSET(186, 86), color_main);

                GLCD_DrawLine(OFFSET(202, 63), OFFSET(212, 73), color_main);
    GLCD_DrawLine(OFFSET(212, 73), OFFSET(204, 85), color_main);
    GLCD_DrawLine(OFFSET(204, 85), OFFSET(195, 75), color_main);
    GLCD_DrawLine(OFFSET(195, 75), OFFSET(202, 63), color_main);

    GLCD_DrawLine(OFFSET(221, 85), OFFSET(218, 60), color_main);
    GLCD_DrawLine(OFFSET(218, 60), OFFSET(232, 68), color_main);
    GLCD_DrawLine(OFFSET(232, 68), OFFSET(220, 76), color_main);
    GLCD_DrawLine(OFFSET(220, 76), OFFSET(235, 82), color_main);

    // White lines
    GLCD_DrawLine(OFFSET(0, 46), OFFSET(35, 56), color_secondary);
    GLCD_DrawLine(OFFSET(35, 56), OFFSET(0, 70), color_secondary);
    GLCD_DrawLine(OFFSET(0, 93), OFFSET(54, 77), color_secondary);
    GLCD_DrawLine(OFFSET(54, 77), OFFSET(0, 121), color_secondary);
    GLCD_DrawLine(OFFSET(291, 0), OFFSET(264, 54), color_secondary);
    GLCD_DrawLine(OFFSET(264, 54), OFFSET(319, 21), color_secondary);
    GLCD_DrawLine(OFFSET(319, 39), OFFSET(288, 58), color_secondary);
    GLCD_DrawLine(OFFSET(288, 68), OFFSET(319, 61), color_secondary);
}

void Game_Start() {
    uint32_t joystick_state;
    int x_pos = 0, y_pos = 0;  // Starting position

    transition_screen(0x0000, rand() % 6, 0);
    GLCD_SetBackColor(Black);
    GLCD_SetTextColor(White);

    GLCD_DisplayString(20, 0, 0, "              Move the Joystick to Play             ");
    GLCD_DisplayString(21, 0, 0, "                  [SELECT TO START]                 ");
    GLCD_DisplayString(24, 0, 0, "          Survive the attack of the red balls       ");
    GLCD_DrawBox(30, 140, 260, 80, White);

                draw_connected_lines(x_pos, y_pos, 0);
```

```
    delay(50);

    while((joystick_state & KBD_SELECT) == 0) {
                                if(tickCount %10 == 0){
                                draw_connected_lines(x_pos, y_pos, 0);
        draw_connected_lines(x_pos, y_pos, 1);
                                }


        joystick_state = get_button();
        incrementTicks();
        snprintf(buf, sizeof(buf), "Ticks: %llu", tickCount);
        GLCD_DisplayString(32, 0, 0, (unsigned char *)buf);

    }
    Game_Script_Init();
}

void Game_Update(uint32_t ticks){
        Game_Script_Update(ticks);
        Game_Script_TimerTick();
}

void MainMenu_Input(uint32_t key){
        if (key == KBD_SELECT){
                curr_state = menu_select +1;
        } else{
                MainMenuScript_Input(key);
        }
}

void Gallery_Input(uint32_t key){
  switch(key){
    case KBD_LEFT:
      currentImage--;
      if (currentImage < 0){
        currentImage = gallerySize - 1;
      }
      break;
    case KBD_RIGHT:
      currentImage++;
      if (currentImage >= gallerySize){
        currentImage = 0;
      }
      break;
    case KBD_UP:
      curr_state = STATE_MAIN_MENU;
      break;
    default:
      break;
  }
        delay(15);
}


void simple_input(uint32_t key){
  if (key == KBD_UP){
    curr_state = STATE_MAIN_MENU;
  }
        if (curr_state != STATE_MP3_PLAYER){
                delay(15);
        }

}

//the game input handler
// handles the joystick movement and the button press
void game_input(uint32_t key){
  Game_Script_Input(key);
}


//Thread Declarations and priority configurations
void GameTimer_handler(void const *argument);
```

```
387 //void UpdateLoopThread(void const *argument);
388
389 osThreadId GameTimerThread_ID;
390 //osThreadId UpdateLoopThread_ID;
391 osThreadId t_main_ID;
392
393 osThreadDef(GameTimer_handler, osPriorityAboveNormal, 128, 0);
394 //osThreadDef(UpdateLoopThread, osPriorityNormal, 128, 0);
395
396
397
398
399 void GameTimer_handler(void const *argument){
400         //GLCD_Clear(Red);
401         while(1){
402                 osSignalWait(0x01, osWaitForever);
403                 GLCD_Clear(DarkGreen);
404                 Game_Script_TimerTick();
405                 //osThreadYield();
406         }
407 }
408
409 // Timer callback functions to trigger each process based on its period
410 void TimerCallbackA(void const *param) {
411     osSignalSet(GameTimerThread_ID, 0x01);  // Signal Process A to start
412 }
413 osTimerDef(GameTimer, TimerCallbackA);
414
415
416
417
418
419 void UpdateLoopThread(){
420         uint32_t joystick_state;  // Get Joystick state
421         GLCD_Clear(Blue);
422         for (;;)
423         {
424                 joystick_state = get_button();  // Get Joystick state
425                 //osSignalWait(0x02, osWaitForever);
426                 //GLCD_Clear(Yellow);
427                 if (curr_state != prev_state)
428                 {
429                         if (States[prev_state].exit != NULL && prev_state != -1)
430                         {
431                                 States[prev_state].exit();
432                         }
433                         //GLCD_Clear(DarkGreen);
434                         States[curr_state].init();
435                         prev_state = curr_state;
436                         //GLCD_Clear(Red);
437                 }
438                 incrementTicks();
439                 States[curr_state].update(tickCount);
440                 States[curr_state].input(joystick_state);
441                 //osThreadYield();
442         }
443 }
444
445
446
447
448
449
450 // Main function
451 int main(void) {
452         //int i,j,k,l;
453
454         //t_main_ID = osThreadGetId();
455         //osThreadSetPriority(t_main_ID, osPriorityHigh);
456
457         //SystemInit();
458         KBD_Init();
459         Mute = TRUE;
460
```

```
461
462         #ifdef __USE_LCD
463         GLCD_Init();
464         GLCD_Clear(White);
465         transition_screen(Black, PATTERN_LEFT_TO_RIGHT,  0);            //Black Transition - Random Pattern
466         GLCD_SetBackColor(Black);
467         GLCD_SetTextColor(White);
468         GLCD_DisplayStringCutout(0, 0, 1 , "COE718 Final Project", White);
469         GLCD_DisplayStringCutout(4, 0, 0 , "    Created by Victor Do - 501137174", White);
470         #endif
471
472
473         #ifdef __USE_LCD
474         //GLCD_Clear(Blue);
475         transition_screen(Blue, rand() % 6,  0);
476         GLCD_DisplayStringCutout(0, 0, 1 , "COE718 Final Project", White);
477         GLCD_DisplayStringCutout(4, 0, 0 , "    Created by Victor Do - 501137174", White);
478         #endif
479
480         #ifdef __USE_LCD
481         //GLCD_Clear(Red);
482         transition_screen(Red, rand() % 6,  0);
483         GLCD_DisplayStringCutout(0, 0, 1 , "COE718 Final Project", White);
484         GLCD_DisplayStringCutout(4, 0, 0 , "    Created by Victor Do - 501137174", White);
485         #endif
486
487         #ifdef __USE_LCD
488         //GLCD_Clear(Green);
489         transition_screen(Green, rand() % 6,  0);
490         GLCD_DisplayStringCutout(0, 0, 1 , "COE718 Final Project", White);
491         GLCD_DisplayStringCutout(4, 0, 0 , "    Created by Victor Do - 501137174", White);
492         #endif
493
494         //USBAudio_Init();
495
496
497         States[STATE_MAIN_MENU].init = MainMenu_Init;
498         States[STATE_MAIN_MENU].update = MainMenu_Update;
499         States[STATE_MAIN_MENU].input = MainMenu_Input;
500         States[STATE_MAIN_MENU].exit = NULL;
501
502         States[STATE_GALLERY].init = Gallery_Start;
503         States[STATE_GALLERY].update = Gallery_Update;
504         States[STATE_GALLERY].input = Gallery_Input;
505         States[STATE_GALLERY].exit = NULL;
506
507         States[STATE_MP3_PLAYER].init = MP3_Player_Start;
508         States[STATE_MP3_PLAYER].update = MP3_Player_Update;
509         States[STATE_MP3_PLAYER].input = simple_input;
510         States[STATE_MP3_PLAYER].exit = MP3_Player_Exit;
511
512         States[STATE_GAME].init = Game_Start;
513         States[STATE_GAME].update = Game_Update;
514         States[STATE_GAME].input = game_input;
515         States[STATE_GAME].exit = GAME_Exit;
516
517         // Create the mutex
518                 //    loggerMutex = osMutexCreate(osMutex(loggerMutex));
519     //if (loggerMutex == NULL) {
520         // Handle error: Failed to create the mutex
521     //}
522
523     //Init_Threads();        // Create and initialize threads
524
525         #ifdef __USE_LCD
526         //GLCD_Clear(Yellow);
527         transition_screen(Yellow, rand() % 6,  0);
528         GLCD_DisplayStringCutout(0, 0, 1 , "COE718 Final Project", White);
529         GLCD_DisplayStringCutout(4, 0, 0 , "    Created by Victor Do - 501137174", White);
530         #endif
531         //start UpdateLoopThread
532
533
534
```

```
        #ifdef __USE_LCD
        //GLCD_Clear(Magenta);
        transition_screen(Magenta, rand() % 6,  0);
        GLCD_DisplayStringCutout(0, 0, 1 , "COE718 Final Project", White);
        GLCD_DisplayStringCutout(4, 0, 0 , "    Created by Victor Do - 501137174", White);
        #endif

        //delete own thread
        //osDelay(osWaitForever);



        //osThreadSetPriority(t_main_ID, osPriorityBelowNormal);

    // Round-robin scheduling: Execute tasks and yield control
    /*while (1) {
                        osThreadYield();
        //UpdateLoopThread(NULL);
                                //GLCD_Clear(Magenta);
                */

        #ifdef __USE_LCD
        //GLCD_Clear(Red);
        transition_screen(Red, rand() % 6,  0);
        GLCD_DisplayStringCutout(0, 0, 1 , "COE718 Final Project", White);
        GLCD_DisplayStringCutout(4, 0, 0 , "    Created by Victor Do - 501137174", White);
        #endif

        //osKernelInitialize();  // Initialize RTOS
        //t_main_ID = osThreadGetId();
        //osThreadSetPriority(t_main_ID, osPriorityHigh);

        //GameTimerThread_ID = osThreadCreate(osThread(GameTimer_handler), NULL);
   //if (!GameTimerThread_ID) return -1;

        //osTimerId GameTimer = osTimerCreate(osTimer(GameTimer), osTimerPeriodic, (void *)1);
        //GLCD_Clear(Green);
        transition_screen(Green, rand() % 6,  0);
        GLCD_DisplayStringCutout(0, 0, 1 , "COE718 Final Project", White);
        GLCD_DisplayStringCutout(4, 0, 0 , "    Created by Victor Do - 501137174", White);

        //osKernelStart();       // Start thread execution

        //osTimerStart(GameTimer, 2000);

        #ifdef __USE_LCD
        //GLCD_Clear(Yellow);
        transition_screen(Yellow, rand() % 6,  0);
        GLCD_DisplayStringCutout(0, 0, 1 , "COE718 Final Project", White);
        GLCD_DisplayStringCutout(4, 0, 0 , "    Created by Victor Do - 501137174", White);
        #endif
        //osThreadTerminate(t_main_ID);

        #ifdef __USE_LCD
        //GLCD_Clear(Magenta);
        transition_screen(Magenta, rand() % 6,  0);
        GLCD_DisplayStringCutout(0, 0, 1 , "COE718 Final Project", White);
        GLCD_DisplayStringCutout(4, 0, 0 , "    Created by Victor Do - 501137174", White);
        #endif
        osThreadSetPriority(t_main_ID, osPriorityNormal);
        while (1) {
                UpdateLoopThread();
        }                               // Infinite loop to keep main active


}
```

C. *The following is my code for the Main Menu*

```
// MainMenu Code
#include <stdio.h>
#include "cmsis_os.h" // CMSIS RTOS header file
#include "LPC17xx.h"
```

```c
#include "GLCD.h"
#include "MainMenu.h"
#include "KBD.h"

#define ICON_SIZE 64
#define ICON_OUTLINE_SIZE 72

//#include "Images/darkerBG.h"
//#include "Images/gameIcon.h"
//#include "Images/gallery.h"
//#include "Images/musical-note.h"

//#include "darkerBG.c"

extern unsigned char MAINMENU_BG_pixel_data[];
extern unsigned char GALLERY_ICON_pixel_data[];
extern unsigned char MUSIC_ICON_pixel_data[];
extern unsigned char GAME_ICON_pixel_data[];


int menu_select = 0;
int needs_update = 1;

void MainMenu_Init(void) {
  GLCD_Bitmap(0, 0, 320, 240, MAINMENU_BG_pixel_data);

        GLCD_DisplayStringCutout(0, 0, 1 , "COE718 Final Project", White);
        GLCD_DisplayStringCutout(4, 0, 0 , "    Created by Victor Do - 501137174", White);

  GLCD_BitmapCutout( 43, 80, ICON_SIZE, ICON_SIZE, GALLERY_ICON_pixel_data, White);
  GLCD_BitmapCutout(128, 80, ICON_SIZE, ICON_SIZE, MUSIC_ICON_pixel_data, White);
  GLCD_BitmapCutout(213, 80, ICON_SIZE, ICON_SIZE, GAME_ICON_pixel_data, White);

        //GLCD_Bitmap( 43, 80, ICON_SIZE, ICON_SIZE, GALLERY_ICON_pixel_data);
  //GLCD_Bitmap(128, 80, ICON_SIZE, ICON_SIZE, MUSIC_ICON_pixel_data);
  //GLCD_Bitmap(213, 80, ICON_SIZE, ICON_SIZE, GAME_ICON_pixel_data);
}

void MainMenu_Update(uint32_t ticks) {
        int i;
        //GLCD_Clear(White);

  if(needs_update) {
                // To redraw just a 50x50 region starting at (100,100) of a background bitmap
                GLCD_Bitmap_Region(0, 0, 320, 240, MAINMENU_BG_pixel_data, 39, 76, 242, 72);

    for(i = 0; i < 3; i++) {
      if(i == menu_select) {
        GLCD_DrawBox((43-4)+i*85, 80-4, ICON_OUTLINE_SIZE, ICON_OUTLINE_SIZE, White);
      }
    }

    if (menu_select == 0) {
                          //GLCD_BitmapCutout( 43, 80, ICON_SIZE, ICON_SIZE, GALLERY_ICON_pixel_data, White
                              );
                            GLCD_Bitmap( 43, 80, ICON_SIZE, ICON_SIZE, GALLERY_ICON_pixel_data);
        GLCD_BitmapCutout(128, 80, ICON_SIZE, ICON_SIZE, MUSIC_ICON_pixel_data, White);
        GLCD_BitmapCutout(213, 80, ICON_SIZE, ICON_SIZE, GAME_ICON_pixel_data, White);

        GLCD_DisplayString(7, 0, 1 , "    >  PICTURES  <    ");
    } else if (menu_select == 1) {
                              //GLCD_BitmapCutout(128, 80, ICON_SIZE, ICON_SIZE, MUSIC_ICON_pixel_data,
                                  White);
                              GLCD_Bitmap(128, 80, ICON_SIZE, ICON_SIZE, MUSIC_ICON_pixel_data);
        GLCD_BitmapCutout( 43, 80, ICON_SIZE, ICON_SIZE, GALLERY_ICON_pixel_data, White);
        GLCD_BitmapCutout(213, 80, ICON_SIZE, ICON_SIZE, GAME_ICON_pixel_data, White);

        GLCD_DisplayString(7, 0, 1 , "   >  MP3 PLAYER  <   ");
    } else if (menu_select == 2) {
                              //GLCD_BitmapCutout(213, 80, ICON_SIZE, ICON_SIZE, GAME_ICON_pixel_data,
                                  White);
                              GLCD_Bitmap(213, 80, ICON_SIZE, ICON_SIZE, GAME_ICON_pixel_data);
        GLCD_BitmapCutout( 43, 80, ICON_SIZE, ICON_SIZE, GALLERY_ICON_pixel_data, White);
        GLCD_BitmapCutout(128, 80, ICON_SIZE, ICON_SIZE, MUSIC_ICON_pixel_data, White);
```

```
76
77          GLCD_DisplayString(7, 0, 1 , "      >  GAME  <      ");
78      }
79
80                GLCD_DrawBox(0, 167, 320, 26, 0x0000);
81          GLCD_DrawBox(0, 165, 320, 2, 0x0000);
82
83    }
84    needs_update = 0;
85  }
86  void delayM (int cnt) {
87    cnt <<= 18;
88    while (cnt--);
89  }
90
91  void MainMenuScript_Input(uint32_t key) {
92    if(key & KBD_LEFT) {
93      menu_select--;
94      if(menu_select < 0) menu_select = 2;
95                  needs_update = 1;
96    }
97    else if(key & KBD_RIGHT) {
98      menu_select++;
99      if(menu_select > 2) menu_select = 0;
100                 needs_update = 1;
101   }
102         else if(key & KBD_UP) {
103                 needs_update = 1;
104   }
105         else if(key & KBD_RIGHT) {
106                 needs_update = 1;
107   }
108         delayM(15);
109 }
```

D. The following is the game script code

```
1  #include <stdio.h>
2  #include "cmsis_os.h" // CMSIS RTOS header file
3  #include "LPC17xx.h"
4  #include "GLCD.h"
5  #include "GameScript.h"
6  #include "KBD.h"
7  #include "MainMenu.h"
8  #include <stdint.h>
9  #include <stdlib.h>
10 #include <math.h>
11 #include "main.h"
12
13 // Add missing PI definition
14 #ifndef M_PI
15 #define M_PI 3.14159265358979323846f
16 #endif
17
18 // Helper macros for min/max since we can't use fmin/fmax
19 #define MIN(a, b) ((a) < (b) ? (a) : (b))
20 #define MAX(a, b) ((a) > (b) ? (a) : (b))
21
22 #define MAP_WIDTH 960
23 #define MAP_HEIGHT 720
24 #define SCREEN_WIDTH 320
25 #define SCREEN_HEIGHT 240
26 #define MAX_ENEMIES 32
27 #define MAX_PROJECTILES 16
28 #define MAX_UPGRADES 5
29 #define MAX_COINS 10
30
31 #define OBSTACLE_SIZE 144
32 #define CENTER_GRID_X 320
33 #define CENTER_GRID_Y 240
34 #define SCREEN_CENTER_X (SCREEN_WIDTH / 2)
35 #define SCREEN_CENTER_Y (SCREEN_HEIGHT / 2)
36
```

```c
#define MAX_PROJECTILE_SPEED 4.0f
#define MAX_PROJECTILE_RADIUS 50
#define MAX_PROJECTILE_LIFETIME 100 // 10 seconds
#define MIN_FIRE_RATE 3              // milliseconds between shots

// Add coin-specific defines
#define COIN_RADIUS 5
#define COIN_SPAWN_RATE 20 // 5 seconds between coin spawns
#define COIN_LIFETIME 20  // Coins disappear after 15 seconds
#define COIN_SCORE 100

#define PLAYER_RADIUS 10
#define MAX_ENEMY_RADIUS 10


// defines for upgrades
#define UPGRADE_RADIUS 8
#define UPGRADE_SPAWN_RATE 20 // 10 seconds between upgrade spawns
#define UPGRADE_LIFETIME 20  // Upgrades disappear after 10 seconds
#define MULTI_SHOT_MAX 3        // Maximum number of simultaneous projectiles

// Game State Structures
typedef struct
{
    int x, y;           // Position
    int health;         // Hearts
    int score;          // Current score
    float speed;        // Movement speed
    int invulnerable;   // Invulnerability timer
    int last_hit_time;  // Time since last hit
    int last_shot_time; // Track last shot time for fire rate
    int multi_shot;     // Number of projectiles fired at once

} Player;

typedef struct
{
    int screen_x;
    int screen_y;
} ScreenPos;

typedef struct
{
    int x, y; // World position
    int active;
    float speed;
    ScreenPos last_pos; // Last screen position
} Enemy;

typedef struct
{
    int x, y;
    int active;
    // float speed;
    float dx, dy; // Direction vector
    int lifetime;
    ScreenPos last_pos;
} Projectile;

typedef struct
{
    int x, y;
    int type; // 0: Coin, 1: Speed, 2: Projectile Size, 3: Fire Rate
    int active;
    ScreenPos last_pos;
} Upgrade;

typedef struct
{
    int x, y;
    int active;
    int spawn_time; // When the coin was spawned
    ScreenPos last_pos;
} Coin;
```

```
111
112 // Global Game Variables
113 Player player;
114 Enemy enemies[MAX_ENEMIES];
115 Projectile projectiles[MAX_PROJECTILES];
116 Upgrade upgrades[MAX_UPGRADES];
117 Coin coins[MAX_COINS];
118 uint16_t MAX_DIFFICULTY;
119 static int last_score = -1;
120 static int last_health = -1;
121
122 Enemy *closest_enemy = NULL;
123 float closest_enemy_distance = INFINITY;
124
125 int game_time = 0;
126 int enemy_spawn_rate = 30; // milliseconds
127 int last_enemy_spawn = 0;
128 int last_coin_spawn = 0;
129 int difficulty_multiplier = 1;
130 int last_upgrade_spawn = 0;
131 int timerCounter = 0;
132
133 float PROJECTILE_SPEED = 4.0f;
134 uint8_t PROJECTILE_RADIUS = 5;
135 uint32_t PROJECTILE_LIFETIME = 20; // 2 seconds
136 uint32_t FIRE_RATE = 20;           // milliseconds between shots
137 uint8_t ENEMY_RADIUS = 10;
138 char score_str[20];
139
140 // Initialize Game State
141 void Game_Script_Init(void)
142 {
143                 snprintf(score_str, sizeof(score_str), "Score:␣%d\0\0\0\0\0\0\0\0\0\0\0\0", 0);
144                 last_score = -1;
145                 last_health = -1;
146     // Initialize Player
147     int i = 0;
148     player.x = MAP_WIDTH / 2;
149     player.y = MAP_HEIGHT / 2;
150     player.health = 3;
151     player.score = 0;
152     player.speed = 4.0;
153     player.invulnerable = 0;
154     player.multi_shot = 1; // Initialize with single shot
155
156                 MAX_DIFFICULTY = player.speed - 1;
157
158     timerCounter = 0;
159     ENEMY_RADIUS = 10;
160
161     game_time = 0;
162     enemy_spawn_rate = 30; // milliseconds
163     last_enemy_spawn = 0;
164     last_coin_spawn = 0;
165     difficulty_multiplier = 1;
166     last_upgrade_spawn = 0;
167
168     PROJECTILE_SPEED = 4.0f;
169     PROJECTILE_RADIUS = 5;
170     PROJECTILE_LIFETIME = 100; // 2 seconds
171     FIRE_RATE = 100;           // milliseconds between shots
172     ENEMY_RADIUS = 10;
173
174     // Clear Enemies
175     for (i = 0; i < MAX_ENEMIES; i++)
176     {
177         enemies[i].active = 0;
178     }
179
180     // Clear Projectiles
181     for (i = 0; i < MAX_PROJECTILES; i++)
182     {
183         projectiles[i].active = 0;
184     }
```

```
185
186     // Clear Upgrades
187     for (i = 0; i < MAX_UPGRADES; i++)
188     {
189         upgrades[i].active = 0;
190     }
191
192     // Clear Coins
193     for (i = 0; i < MAX_COINS; i++)
194     {
195         coins[i].active = 0;
196     }
197
198     // Draw Initial Screen Elements
199     GLCD_Clear(Black);
200     GLCD_DrawCircle(SCREEN_WIDTH / 2, SCREEN_HEIGHT / 2, PLAYER_RADIUS, Blue); // Player
201     //GLCD_DisplayStringCutout(0, 0, 1, "Score: 0", White);
202                 //GLCD_DisplayString(0, 0, 1, "Score: 0");
203
204     // Draw Heart Icons
205     for (i = 0; i < player.health; i++)
206     {
207         GLCD_DrawLine(305 - i * 20, 5, 305 - i * 20, 15, Red);
208         GLCD_DrawLine(310 - i * 20, 10, 300 - i * 20, 10, Red);
209     }
210 }
211
212 // Function to spawn a new projectile
213 void SpawnProjectile(void)
214 {
215     int i, shot;
216     int currentTime = game_time;
217     float base_dx, base_dy, length;
218     // Check fire rate
219     if (currentTime - player.last_shot_time > FIRE_RATE)
220     {
221         return;
222     }
223
224     // Only shoot if there's an enemy
225     if (closest_enemy == NULL)// || !closest_enemy->active)
226     {
227         return;
228     }
229
230     base_dx = closest_enemy->x - player.x;
231     base_dy = closest_enemy->y - player.y;
232     length = sqrtf(base_dx * base_dx + base_dy * base_dy);
233
234     // For each shot in multi-shot
235     for (shot = 0; shot < player.multi_shot; shot++)
236     {
237         for (i = 0; i < MAX_PROJECTILES; i++)
238         {
239             if (!projectiles[i].active)
240             {
241                 float angle_offset = (shot - (player.multi_shot - 1) / 2.0f) * 15.0f; // 15 degree spread
242                 float rad_angle = angle_offset * M_PI / 180.0f;
243
244                 // Rotate the direction vector
245                 float dx = base_dx * cosf(rad_angle) - base_dy * sinf(rad_angle);
246                 float dy = base_dx * sinf(rad_angle) + base_dy * cosf(rad_angle);
247
248                 // Normalize and apply speed
249                 if (length > 0)
250                 {
251                     projectiles[i].dx = (dx / length) * PROJECTILE_SPEED;
252                     projectiles[i].dy = (dy / length) * PROJECTILE_SPEED;
253                 }
254
255                 projectiles[i].x = player.x;
256                 projectiles[i].y = player.y;
257                 projectiles[i].active = 1;
258                 projectiles[i].lifetime = currentTime + PROJECTILE_LIFETIME;
```

```
                projectiles[i].last_pos.screen_x = SCREEN_CENTER_X;
                projectiles[i].last_pos.screen_y = SCREEN_CENTER_Y;
                break;
            }
        }
    }
    player.last_shot_time = currentTime;
}

// Spawn Enemies Near Screen Edges
void SpawnEnemy(void)
{
    int i = 0;
    for (i = 0; i < MAX_ENEMIES; i++)
    {
        if (!enemies[i].active)
        {
            // Randomly choose side to spawn
            int side = rand() % 4;
            switch (side)
            {
            case 0: // Top
                enemies[i].x = player.x + (rand() % SCREEN_WIDTH) - SCREEN_WIDTH / 2;
                enemies[i].y = player.y - SCREEN_HEIGHT / 2 - 20;
                break;
            case 1: // Bottom
                enemies[i].x = player.x + (rand() % SCREEN_WIDTH) - SCREEN_WIDTH / 2;
                enemies[i].y = player.y + SCREEN_HEIGHT / 2 + 20;
                break;
            case 2: // Left
                enemies[i].x = player.x - SCREEN_WIDTH / 2 - 20;
                enemies[i].y = player.y + (rand() % SCREEN_HEIGHT) - SCREEN_HEIGHT / 2;
                break;
            case 3: // Right
                enemies[i].x = player.x + SCREEN_WIDTH / 2 + 20;
                enemies[i].y = player.y + (rand() % SCREEN_HEIGHT) - SCREEN_HEIGHT / 2;
                break;
            }
            enemies[i].speed = 1.0 * difficulty_multiplier;
            enemies[i].active = 1;
            break;
        }
    }
}

// Function to convert world coordinates to screen coordinates
void WorldToScreen(int worldX, int worldY, int *screenX, int *screenY)
{
    *screenX = worldX - (player.x - SCREEN_CENTER_X);
    *screenY = worldY - (player.y - SCREEN_CENTER_Y);
}

int CheckObstacleCollision(int x, int y, int radius)
{
    // Define obstacle positions in world space
    int center_grid_left = CENTER_GRID_X - OBSTACLE_SIZE / 2 - 64;
    int center_grid_right = CENTER_GRID_X + MAP_WIDTH / 3 - OBSTACLE_SIZE / 2 + 64;
    int center_grid_top = CENTER_GRID_Y - OBSTACLE_SIZE / 2 - 64;
    int center_grid_bottom = CENTER_GRID_Y + MAP_HEIGHT / 3 - OBSTACLE_SIZE / 2 + 64;

    // Check collision with each obstacle (including radius in calculation)
    // Top-left obstacle
    if (x + radius > center_grid_left && x - radius < center_grid_left + OBSTACLE_SIZE &&
        y + radius > center_grid_top && y - radius < center_grid_top + OBSTACLE_SIZE)
        return 1;

    // Top-right obstacle
    if (x + radius > center_grid_right && x - radius < center_grid_right + OBSTACLE_SIZE &&
        y + radius > center_grid_top && y - radius < center_grid_top + OBSTACLE_SIZE)
        return 1;

    // Bottom-left obstacle
    if (x + radius > center_grid_left && x - radius < center_grid_left + OBSTACLE_SIZE &&
        y + radius > center_grid_bottom && y - radius < center_grid_bottom + OBSTACLE_SIZE)
```

```
        return 1;

    // Bottom-right obstacle
    if (x + radius > center_grid_right && x - radius < center_grid_right + OBSTACLE_SIZE &&
        y + radius > center_grid_bottom && y - radius < center_grid_bottom + OBSTACLE_SIZE)
        return 1;

    return 0;
}

// New function to spawn coins
void SpawnCoin(void)
{
    int i;
    for (i = 0; i < MAX_COINS; i++)
    {
        if (!coins[i].active)
        {
            // Spawn coin in a random location near the player
            // but not too close to make it challenging
            int angle = rand() % 360;
            float radius = 100 + (rand() % 100); // Between 100-200 pixels from player

            // Use cosf and sinf for single-precision float calculations
            coins[i].x = player.x + (int)(radius * cosf(angle * M_PI / 180.0f));
            coins[i].y = player.y + (int)(radius * sinf(angle * M_PI / 180.0f));

            // Ensure coin is within map bounds using our MAX/MIN macros
            coins[i].x = MAX(COIN_RADIUS, MIN(MAP_WIDTH - COIN_RADIUS, coins[i].x));
            coins[i].y = MAX(COIN_RADIUS, MIN(MAP_HEIGHT - COIN_RADIUS, coins[i].y));

            // Don't spawn if would collide with obstacle
            if (CheckObstacleCollision(coins[i].x, coins[i].y, COIN_RADIUS))
            {
                continue;
            }

            coins[i].active = 1;
            coins[i].spawn_time = game_time;
            coins[i].last_pos.screen_x = -1; // Force initial draw
            coins[i].last_pos.screen_y = -1;
            break;
        }
    }
}
// Function to spawn upgrades
void SpawnUpgrade(void)
{
    int i;
    for (i = 0; i < MAX_UPGRADES; i++)
    {
        if (!upgrades[i].active)
        {
            // Spawn upgrade in random location near the player
            int angle = rand() % 360;
            float radius = 150 + (rand() % 100); // Between 150-250 pixels from player

            upgrades[i].x = player.x + (int)(radius * cosf(angle * M_PI / 180.0f));
            upgrades[i].y = player.y + (int)(radius * sinf(angle * M_PI / 180.0f));

            // Ensure upgrade is within map bounds
            upgrades[i].x = MAX(UPGRADE_RADIUS, MIN(MAP_WIDTH - UPGRADE_RADIUS, upgrades[i].x));
            upgrades[i].y = MAX(UPGRADE_RADIUS, MIN(MAP_HEIGHT - UPGRADE_RADIUS, upgrades[i].y));

            // Don't spawn if would collide with obstacle
            if (CheckObstacleCollision(upgrades[i].x, upgrades[i].y, UPGRADE_RADIUS))
            {
                continue;
            }

            // Randomly choose upgrade type
            upgrades[i].type = rand() % 5;
            upgrades[i].active = 1;
            break;
```

```
        }
    }
}

// Function to apply upgrade effects
void ApplyUpgrade(int type)
{
    switch (type)
    {
    case 0: // Fire rate upgrade
        if (FIRE_RATE > MIN_FIRE_RATE)
        {
            FIRE_RATE = MAX(MIN_FIRE_RATE, FIRE_RATE - 5); // Decrease by 5ms, but not below minimum
        }
        break;

    case 1:                                                 // Speed upgrade
        player.speed = MIN(player.speed + 4.0f, 16.0f); // Cap at 16
        break;

    case 2: // Multi-shot upgrade
        if (player.multi_shot < MULTI_SHOT_MAX)
        {
            player.multi_shot++;
                                              if (player.health < 3)
                                                      player.health++;
        }
        break;

    case 3: // Projectile size upgrade
        if (PROJECTILE_RADIUS < MAX_PROJECTILE_RADIUS)
        {
            PROJECTILE_RADIUS = MIN(PROJECTILE_RADIUS + 2, MAX_PROJECTILE_RADIUS);
        }
        break;
    case 4: // Projectile lifetime upgrade
        if (PROJECTILE_LIFETIME < MAX_PROJECTILE_LIFETIME)
        {
            PROJECTILE_LIFETIME = MIN(PROJECTILE_LIFETIME + 10, MAX_PROJECTILE_LIFETIME);
        }
        break;
    }
}

void UpdateCoins(void)
{
    int i;
    int screen_x, screen_y;
    float dx, dy, distance;

    for (i = 0; i < MAX_COINS; i++)
    {
        if (coins[i].active)
        {
            // Clear old position if it was on screen
            if (coins[i].last_pos.screen_x >= -COIN_RADIUS &&
                coins[i].last_pos.screen_x <= SCREEN_WIDTH + COIN_RADIUS &&
                coins[i].last_pos.screen_y >= -COIN_RADIUS &&
                coins[i].last_pos.screen_y <= SCREEN_HEIGHT + COIN_RADIUS)
            {
                GLCD_DrawFilledCircle(
                    coins[i].last_pos.screen_x,
                    coins[i].last_pos.screen_y,
                    COIN_RADIUS,
                    Black);
            }

            // Check if coin has expired
            if (game_time - coins[i].spawn_time > COIN_LIFETIME)
            {
                coins[i].active = 0;
                continue;
            }

```

```
            // Check if player collected the coin
            dx = coins[i].x - player.x;
            dy = coins[i].y - player.y;
            distance = sqrtf(dx * dx + dy * dy); // Using sqrtf instead of sqrt

            if (distance < PLAYER_RADIUS + COIN_RADIUS)
            {
                player.score += COIN_SCORE;
                coins[i].active = 0;
                continue;
            }

            // Draw coin at new screen position
            WorldToScreen(coins[i].x, coins[i].y, &screen_x, &screen_y);
            coins[i].last_pos.screen_x = screen_x;
            coins[i].last_pos.screen_y = screen_y;

            if (screen_x >= -COIN_RADIUS && screen_x <= SCREEN_WIDTH + COIN_RADIUS &&
                screen_y >= -COIN_RADIUS && screen_y <= SCREEN_HEIGHT + COIN_RADIUS)
            {
                // Make coins flash by alternating colors
                uint16_t coin_color = ((game_time / 250) % 2) ? Yellow : White;
                GLCD_DrawFilledCircle(screen_x, screen_y, COIN_RADIUS, coin_color);
            }
        }
    }
}
void UpdateUpgrades(void)
{
    int i;
    int screen_x, screen_y;
    float dx, dy, distance;

    for (i = 0; i < MAX_UPGRADES; i++)
    {
        if (upgrades[i].active)
        {
            // Clear old position if it was on screen
            if (upgrades[i].last_pos.screen_x >= -UPGRADE_RADIUS &&
                upgrades[i].last_pos.screen_x <= SCREEN_WIDTH + UPGRADE_RADIUS &&
                upgrades[i].last_pos.screen_y >= -UPGRADE_RADIUS &&
                upgrades[i].last_pos.screen_y <= SCREEN_HEIGHT + UPGRADE_RADIUS)
            {
                GLCD_DrawFilledCircle(
                    upgrades[i].last_pos.screen_x,
                    upgrades[i].last_pos.screen_y,
                    UPGRADE_RADIUS,
                    Black);
            }

            // Check if player collected the upgrade
            dx = upgrades[i].x - player.x;
            dy = upgrades[i].y - player.y;
            distance = sqrtf(dx * dx + dy * dy);

            if (distance < PLAYER_RADIUS + UPGRADE_RADIUS)
            {
                ApplyUpgrade(upgrades[i].type);
                upgrades[i].active = 0;
                continue;
            }

            // Draw upgrade at new screen position
            WorldToScreen(upgrades[i].x, upgrades[i].y, &screen_x, &screen_y);
            upgrades[i].last_pos.screen_x = screen_x;
            upgrades[i].last_pos.screen_y = screen_y;

            if (screen_x >= -UPGRADE_RADIUS && screen_x <= SCREEN_WIDTH + UPGRADE_RADIUS &&
                screen_y >= -UPGRADE_RADIUS && screen_y <= SCREEN_HEIGHT + UPGRADE_RADIUS)
            {
                // Different colors for different upgrade types
                uint16_t upgrade_color;
                switch (upgrades[i].type)
                {
```

```
                case 0:
                    upgrade_color = Magenta;
                    break; // Fire rate
                case 1:
                    upgrade_color = Cyan;
                    break; // Speed
                case 2:
                    upgrade_color = Green;
                    break; // Multi-shot
                case 3:
                    upgrade_color = Purple;
                    break; // Projectile size
                default:
                    upgrade_color = White;
                }
                GLCD_DrawFilledCircle(screen_x, screen_y, UPGRADE_RADIUS, upgrade_color);
            }
        }
    }
}
void UpdateProjectiles(void)
{
    int i, j;
    int screen_x, screen_y;

    for (i = 0; i < MAX_PROJECTILES; i++)
    {
        if (projectiles[i].active)
        {
            // Clear old position
            if (projectiles[i].last_pos.screen_x >= -PROJECTILE_RADIUS &&
                projectiles[i].last_pos.screen_x <= SCREEN_WIDTH + PROJECTILE_RADIUS &&
                projectiles[i].last_pos.screen_y >= -PROJECTILE_RADIUS &&
                projectiles[i].last_pos.screen_y <= SCREEN_HEIGHT + PROJECTILE_RADIUS)
            {
                GLCD_DrawCircle(
                    projectiles[i].last_pos.screen_x,
                    projectiles[i].last_pos.screen_y,
                    PROJECTILE_RADIUS,
                    Black);
            }

            // Update position
            projectiles[i].x += projectiles[i].dx;
            projectiles[i].y += projectiles[i].dy;

            // Check lifetime
            if (game_time >= projectiles[i].lifetime)
            {
                projectiles[i].active = 0;
                continue;
            }

            // Check collision with obstacles
            if (CheckObstacleCollision(projectiles[i].x, projectiles[i].y, PROJECTILE_RADIUS))
            {
                projectiles[i].active = 0;
                continue;
            }

            // Check collision with enemies
            for (j = 0; j < MAX_ENEMIES; j++)
            {
                if (enemies[j].active)
                {
                    float dx = projectiles[i].x - enemies[j].x;
                    float dy = projectiles[i].y - enemies[j].y;
                    float distance = sqrtf(dx * dx + dy * dy);

                    if (distance < PROJECTILE_RADIUS + ENEMY_RADIUS)
                    {
                        enemies[j].active = 0;
                        projectiles[i].active = 0;
                        player.score += 10;
```

```
                        break;
                    }
                }
            }

            // Draw new position if active
            if (projectiles[i].active)
            {
                WorldToScreen(projectiles[i].x, projectiles[i].y, &screen_x, &screen_y);
                projectiles[i].last_pos.screen_x = screen_x;
                projectiles[i].last_pos.screen_y = screen_y;

                if (screen_x >= -PROJECTILE_RADIUS && screen_x <= SCREEN_WIDTH + PROJECTILE_RADIUS &&
                    screen_y >= -PROJECTILE_RADIUS && screen_y <= SCREEN_HEIGHT + PROJECTILE_RADIUS)
                {
                    GLCD_DrawFilledCircle(screen_x, screen_y, PROJECTILE_RADIUS, Yellow);
                }
            }
        }
    }
}

// Update Game Logic
void Game_Script_Update(uint32_t ticks)
{
    int i = 0;
    float enemy_newX, enemy_newY, dx, dy, distance;
    int player_visible = 1;

    static ScreenPos last_obstacle_pos[4] = {{0, 0}, {0, 0}, {0, 0}, {0, 0}};
                static ScreenPos last_mapBorder_pos = {0,0};
    int screen_x, screen_y;

    // Calculate obstacle positions in the center grid
    int center_grid_left = CENTER_GRID_X - OBSTACLE_SIZE / 2;
    int center_grid_right = CENTER_GRID_X + MAP_WIDTH / 3 - OBSTACLE_SIZE / 2;
    int center_grid_top = CENTER_GRID_Y - OBSTACLE_SIZE / 2;
    int center_grid_bottom = CENTER_GRID_Y + MAP_HEIGHT / 3 - OBSTACLE_SIZE / 2;


    score_str[0] = '\0';

    // Reset closest enemy tracking at the start of each update
    closest_enemy = NULL;
    closest_enemy_distance = INFINITY;

    game_time = ticks;

    // Add automatic firing at the beginning of the update function
    SpawnProjectile(); // Will only fire if enough time has passed since last shot

    // Update projectiles
    UpdateProjectiles();

    // Spawn coins periodically
    if (game_time - last_coin_spawn > COIN_SPAWN_RATE)
    {
        SpawnCoin();
        last_coin_spawn = game_time;
    }
    // Update coins before drawing other elements
    UpdateCoins();

    // Spawn upgrades periodically
    if (game_time - last_upgrade_spawn > UPGRADE_SPAWN_RATE) {
        SpawnUpgrade();
        last_upgrade_spawn = game_time;
    }
    // Update upgrades
    UpdateUpgrades();

    // Clear previous obstacle positions
    for (i = 0; i < 4; i++)
    {
```

```
        if (last_obstacle_pos[i].screen_x != 0 || last_obstacle_pos[i].screen_y != 0)
        {
            // Clear slightly larger area to ensure complete removal
            GLCD_DrawBox(
                last_obstacle_pos[i].screen_x,
                last_obstacle_pos[i].screen_y,
                OBSTACLE_SIZE,
                OBSTACLE_SIZE,
                Black);
        }
    }

    // Draw new obstacle positions
    // Top-left obstacle
    WorldToScreen(center_grid_left - 64, center_grid_top - 64, &screen_x, &screen_y);
    GLCD_DrawBox(screen_x, screen_y, OBSTACLE_SIZE, OBSTACLE_SIZE, White);
    last_obstacle_pos[0].screen_x = screen_x;
    last_obstacle_pos[0].screen_y = screen_y;

    // Top-right obstacle
    WorldToScreen(center_grid_right + 64, center_grid_top - 64, &screen_x, &screen_y);
    GLCD_DrawBox(screen_x, screen_y, OBSTACLE_SIZE, OBSTACLE_SIZE, White);
    last_obstacle_pos[1].screen_x = screen_x;
    last_obstacle_pos[1].screen_y = screen_y;

    // Bottom-left obstacle
    WorldToScreen(center_grid_left - 64, center_grid_bottom + 64, &screen_x, &screen_y);
    GLCD_DrawBox(screen_x, screen_y, OBSTACLE_SIZE, OBSTACLE_SIZE, White);
    last_obstacle_pos[2].screen_x = screen_x;
    last_obstacle_pos[2].screen_y = screen_y;

    // Bottom-right obstacle
    WorldToScreen(center_grid_right + 64, center_grid_bottom + 64, &screen_x, &screen_y);
    GLCD_DrawBox(screen_x, screen_y, OBSTACLE_SIZE, OBSTACLE_SIZE, White);
    last_obstacle_pos[3].screen_x = screen_x;
    last_obstacle_pos[3].screen_y = screen_y;

                // Map
                WorldToScreen(0, 0, &screen_x, &screen_y);
                if (last_mapBorder_pos.screen_x != 0 || last_mapBorder_pos.screen_y != 0)
                        GLCD_DrawBox(last_mapBorder_pos.screen_x, last_mapBorder_pos.screen_y, MAP_WIDTH,
                            MAP_HEIGHT, Black);
    GLCD_DrawBox(screen_x, screen_y, MAP_WIDTH, MAP_HEIGHT, White);
                last_mapBorder_pos.screen_x = screen_x;
                last_mapBorder_pos.screen_y = screen_y;


    // Spawn Enemies if needed
    if (game_time - last_enemy_spawn > enemy_spawn_rate)
    {
        SpawnEnemy();
        last_enemy_spawn = game_time;
    }

    // Update and Draw Enemies
    for (i = 0; i < MAX_ENEMIES; i++)
    {
        if (enemies[i].active)
        {
            // Clear previous enemy position if it was on screen
            if (enemies[i].last_pos.screen_x >= -ENEMY_RADIUS &&
                enemies[i].last_pos.screen_x <= SCREEN_WIDTH + ENEMY_RADIUS &&
                enemies[i].last_pos.screen_y >= -ENEMY_RADIUS &&
                enemies[i].last_pos.screen_y <= SCREEN_HEIGHT + ENEMY_RADIUS)
            {
                // Clear old enemy position
                GLCD_DrawCircle(
                    enemies[i].last_pos.screen_x,
                    enemies[i].last_pos.screen_y,
                    ENEMY_RADIUS,
                    Black);
            }

            // Move towards player
```

```
            dx = (float)(player.x - enemies[i].x);
            dy = (float)(player.y - enemies[i].y);
            distance = sqrtf(dx * dx + dy * dy);

            // Check if this is the closest enemy
            if (distance < closest_enemy_distance)
            {
                closest_enemy = &enemies[i];
                closest_enemy_distance = distance;
            }

            if (distance > 0)
            {
                enemy_newX = enemies[i].x + (dx / distance) * enemies[i].speed;
                enemy_newY = enemies[i].y + (dy / distance) * enemies[i].speed;

                // Check for obstacle collision at new position
                if (!CheckObstacleCollision(enemy_newX, enemy_newY, ENEMY_RADIUS))
                {
                    enemies[i].x = enemy_newX;
                    enemies[i].y = enemy_newY;
                }
                else
                {
                    // If collision detected, try moving only horizontally or vertically
                    if (!CheckObstacleCollision(enemy_newX, enemies[i].y, ENEMY_RADIUS))
                    {
                        enemies[i].x = enemy_newX; // Horizontal movement ok
                    }
                    else if (!CheckObstacleCollision(enemies[i].x, enemy_newY, ENEMY_RADIUS))
                    {
                        enemies[i].y = enemy_newY; // Vertical movement ok
                    }
                    // If both blocked, enemy stays in place
                }
            }

            // Convert enemy world position to screen position
            WorldToScreen(enemies[i].x, enemies[i].y, &screen_x, &screen_y);

            // Store current screen position for next frame's clearing
            enemies[i].last_pos.screen_x = screen_x;
            enemies[i].last_pos.screen_y = screen_y;

            // Only draw if enemy would be visible on screen
            if (screen_x >= -ENEMY_RADIUS && screen_x <= SCREEN_WIDTH + ENEMY_RADIUS &&
                screen_y >= -ENEMY_RADIUS && screen_y <= SCREEN_HEIGHT + ENEMY_RADIUS)
            {
                GLCD_DrawCircle(screen_x, screen_y, ENEMY_RADIUS, Red);
            }

            // Check if enemy is touching player and player is not invulnerable
            if (!player.invulnerable &&
                abs(enemies[i].x - player.x) < PLAYER_RADIUS + ENEMY_RADIUS &&
                abs(enemies[i].y - player.y) < PLAYER_RADIUS + ENEMY_RADIUS)
            {
                player.health--;
                player.invulnerable = 1;
                player.last_hit_time = game_time;
            }
        }else{
                                            // Clear old enemy position
                GLCD_DrawCircle(
                    enemies[i].last_pos.screen_x,
                    enemies[i].last_pos.screen_y,
                    ENEMY_RADIUS,
                    Black);
                                }
    }

    // Clear previous player position if invulnerable (for flashing effect)
    if (player.invulnerable)
    {
        if (player_visible == 0)
```

```
        {
            GLCD_DrawCircle(SCREEN_CENTER_X, SCREEN_CENTER_Y, PLAYER_RADIUS, Black);
            player_visible = 1;
        }
        // Draw player (always in center of screen)
        else if (player_visible)
        {
            GLCD_DrawCircle(SCREEN_CENTER_X, SCREEN_CENTER_Y, PLAYER_RADIUS, Yellow);
            player_visible = 0;
        }

        // Update invulnerability
        if (game_time - player.last_hit_time > 5)
        {
            player.invulnerable = 0;
            GLCD_DrawCircle(SCREEN_CENTER_X, SCREEN_CENTER_Y, PLAYER_RADIUS, Blue);
        }
    }

    // Update HUD only when it changes

    if (player.score != last_score)
    {
        // Clear previous score area
        snprintf(score_str, sizeof(score_str), "Score:␣%d", last_score);
        //GLCD_DisplayStringCutout(0, 0, 1, (unsigned char *)score_str, Black);

        // Draw new score
        snprintf(score_str, sizeof(score_str), "Score:␣%d", last_score);
        //GLCD_DisplayStringCutout(0, 0, 1, (unsigned char *)score_str, White);
                                GLCD_DisplayString(0, 0, 1, (unsigned char *)score_str);
        last_score = player.score;
    }

    if (player.health != last_health)
    {
        // Clear previous hearts area
        for (i = 2; i >= player.health; i--)
        {
            GLCD_DrawLine(305 - i * 20, 5, 305 - i * 20, 15, Black);
            GLCD_DrawLine(310 - i * 20, 10, 300 - i * 20, 10, Black);
        }

        // Draw Heart Icons
        for (i = 0; i < player.health; i++)
        {
            GLCD_DrawLine(305 - i * 20, 5, 305 - i * 20, 15, Red);
            GLCD_DrawLine(310 - i * 20, 10, 300 - i * 20, 10, Red);
        }
        last_health = player.health;
    }
                for (i = 0; i < player.health; i++)// Draw Heart Icons

                {
                                GLCD_DrawLine(305 - i * 20, 5, 305 - i * 20, 15, Red);
                                GLCD_DrawLine(310 - i * 20, 10, 300 - i * 20, 10, Red);
                }
}

// Handle Player Input
void Game_Script_Input(uint32_t key)
{
    int new_x = player.x;
    int new_y = player.y;
    float velocityX = player.speed;
    float velocityY = player.speed;

    int dir_x = 0, dir_y = 0;
    if (key & KBD_UP)
    {
        dir_y = -1;
    }
    if (key & KBD_DOWN)
```

```
    {
        dir_y = 1;
    }
    if (key & KBD_LEFT)
    {
        dir_x = -1;
    }
    if (key & KBD_RIGHT)
    {
        dir_x = 1;
    }

    if (dir_x != 0 && dir_y != 0)
    {
        // Normalize vector
        float length = sqrtf(dir_x * dir_x + dir_y * dir_y);
        dir_x /= length;
        dir_y /= length;
    }

    velocityX = player.speed * dir_x;
    velocityY = player.speed * dir_y;


    new_y = player.y + velocityY;
    new_x = player.x + velocityX;

    if (velocityX > 0)
    {
        velocityX *= 0.9f;
    }
    if (velocityY > 0)
    {
        velocityY *= 0.9f;
    }

    // Only update position if it's within bounds and not colliding with obstacles
    if (new_x >= 0 && new_x < MAP_WIDTH &&
        new_y >= 0 && new_y < MAP_HEIGHT &&
        !CheckObstacleCollision(new_x, new_y, PLAYER_RADIUS))
    {
        player.x = new_x;
        player.y = new_y;
    }

    if (player.health <= 0) {
        curr_state = STATE_MAIN_MENU;
    }
}



// Timer Tick Function (Called Every X Seconds by timer)
void Game_Script_TimerTick(void)
{
    if (timerCounter++ >= 250)
    {
        if (difficulty_multiplier < MAX_DIFFICULTY)
            difficulty_multiplier++; // Increase difficulty

        if (enemy_spawn_rate > 10)
        {
            enemy_spawn_rate -= 1; // Spawn enemies faster
        }
        if (ENEMY_RADIUS < MAX_ENEMY_RADIUS)
        {
            ENEMY_RADIUS++; // Increase enemy size
        }
        timerCounter = 0;
    }
    player.score++;
}
```